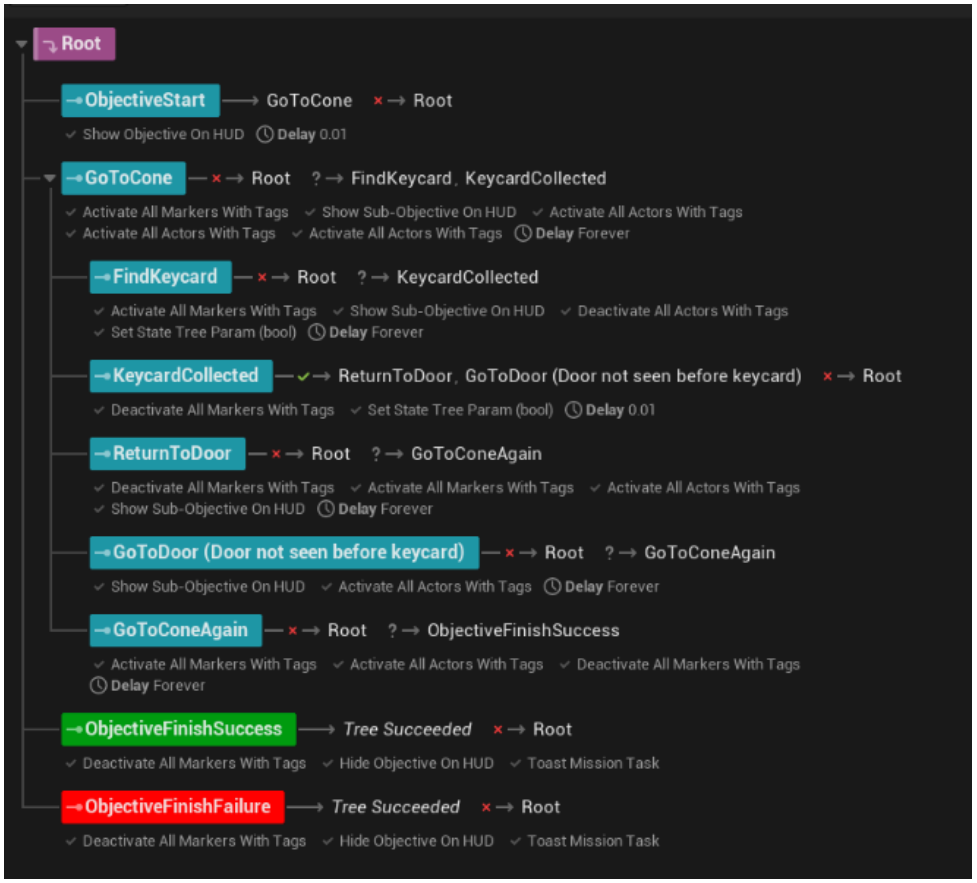
Warwolf Mission System

Unreal Engine State-tree based mission system plugin



Version 1.0

Unreal Engine version: 5.5+

19/12/2024

Table of Contents

- Table of Contents
- Introduction
- Technical Details
- Creator Notes
- Tutorial
  - Integrating the plugin
  - Setting up the component
  - Creating a mission with two simple objectives
    - Setting up our primary objective: Reach a location and interact with an object.
    - Creating the secondary objective: Reach a location and wait 5 seconds
    - Creating the Mission state tree and data asset
  - Setting up the level
  - Running the mission
- Example Content Breakdown
  - ExamplePawn/Character/Actor
  - Game Mode
  - Game State
  - Cone Door
  - Mission Trigger
  - Pickups
  - Item Basket
  - HUD
  - Player State
- Technical Documentation
  - Plugin
    - Warwolf Mission Types
    - Interfaces
      - Warwolf Mission Task Interface
      - Warwolf Mission Interface
      - Warwolf Mission Tag Interface
      - Warwolf Mission UI Interface
    - Tasks
      - Send Objective Start Event
      - Activate All Markers With Tags
      - Deactivate All Markers With Tags
      - Activate All Actors With Tags
      - Deactivate All Actors With Tags
      - Count Gameplay Events (int)
      - Timer
      - Set State Tree Param (bool)
      - Set State Tree Param (float)
      - Set State Tree Param (int32)
      - Toast Mission Task
      - Show Objective On HUD

- Show Sub-Objective On HUD
- Hide Objective On HUD
- Play Level Sequence
- Warwolf Mission Task Helper
- Warwolf Run Parallel

Gameplay Classes

- Warwolf Mission Data Asset
- Warwolf Mission Manager
- Warwolf Mission Marker
- Warwolf Mission Marker Widget
- Warwolf Mission Component Schema
- Warwolf Mission Tag Component

Introduction

Warwolf Mission System is a state tree-based missions/objective system that utilises the (admittedly awesome) versatility of Unreal's state trees, which allows it to support more or less anything you can throw at it with the right knowledge. This plugin is a framework that gives a good few examples of how missions can be built within state trees and should cover enough for you to create the most common mission types. All tasks and classes are built in C++ to squeeze every little bit of performance from the system, but anything a designer would need to access is readily available from blueprints.
What makes this system great, in my opinion, is that you can modularly build objectives and mix-and-match them into different mission assets, and modularly play missions on different levels as it is all gameplay tag-driven.

In addition to the plugin, an example project (which you can download as a test in the links below) covers how to use most of the state tree tasks included and how to implement some basic UI.

Just a note about the plugin. While it provides a handy toolset, and examples such as UI elements and gameplay tag setup, please refer to the example project and the tutorials to get started if you're unable to understand the plugin documentation. That being said, documentation or not, you will need some understanding of the gameplay tags and state tree systems.

Technical Details

- More than 20 C++ classes within the plugin and another 10+ in the example project
- The plugin is all within one module as no editor module is needed
- Heavily exposed to blueprints, though some C++ may be required for certain tasks (will be covered below)
- Several pre-made state tree tasks to cover basic mission logic, and pre-made widget classes in the example project
- State trees do not support replication, but I have tested replication (with some examples) in the example project

Creator Notes

I will try and update and flesh out this system as much as I can, and I'm always happy to hear feedback if you want to drop an email at: support[at]imake.games. Any bugs, feature requests, or even just asking about something that you didn't understand in this documentation should be sent there, please!

Don't forget to leave a review on whichever storefront you got this from, it helps a lot. Thanks for the support!

Tutorial

This tutorial assumes you have some understanding of the engine already. I am also only going to explain how to set everything up in blueprints.

Integrating the plugin

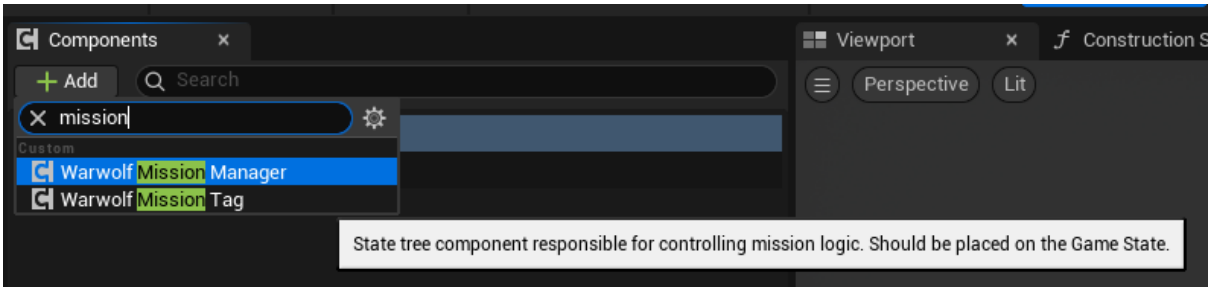
1. Enable the plugin from the ‘Plugins’ menu.



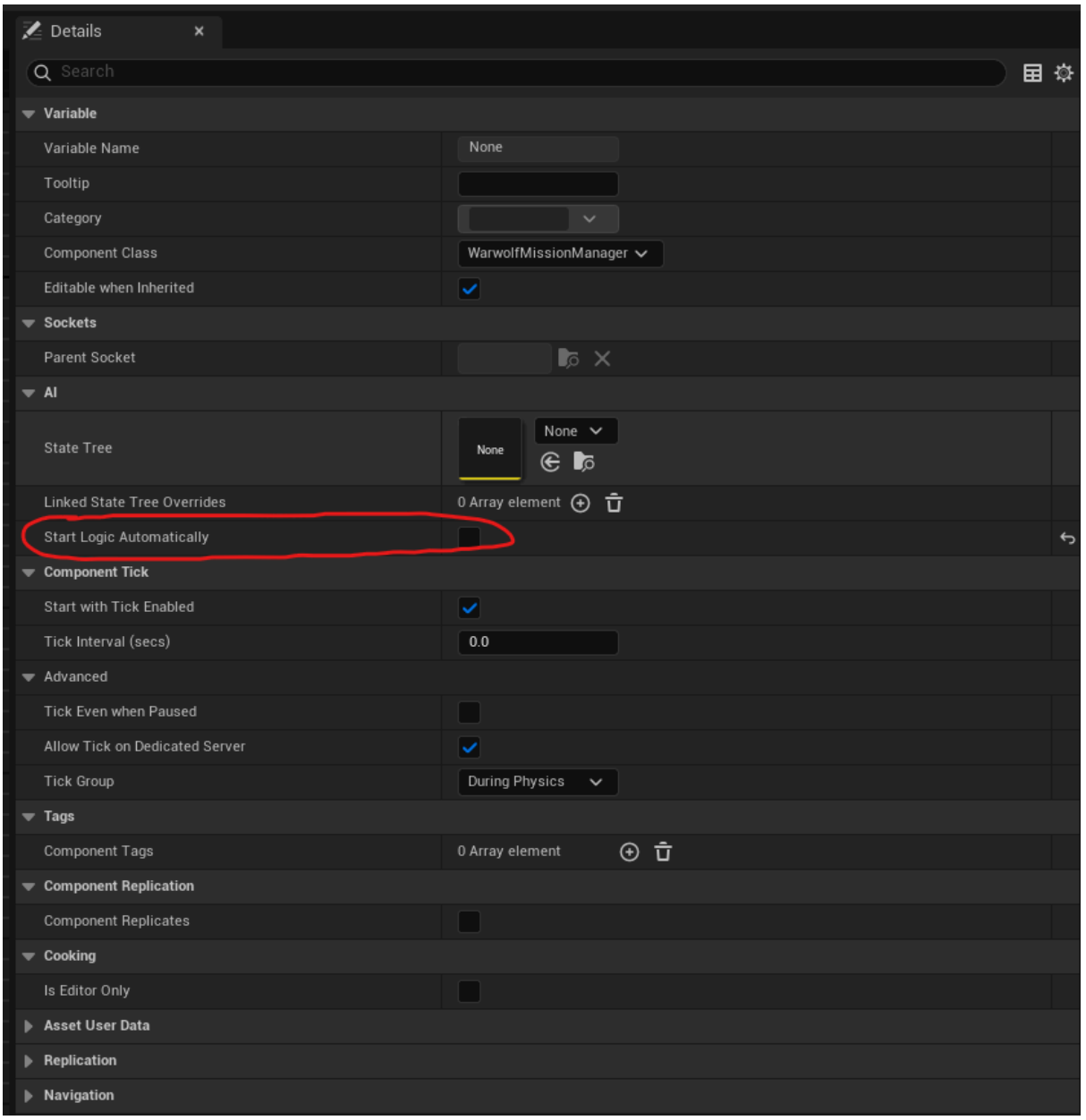
Setting up the component

1. Create a game mode and game state if you haven’t already, and make sure your game state is set on the game mode.

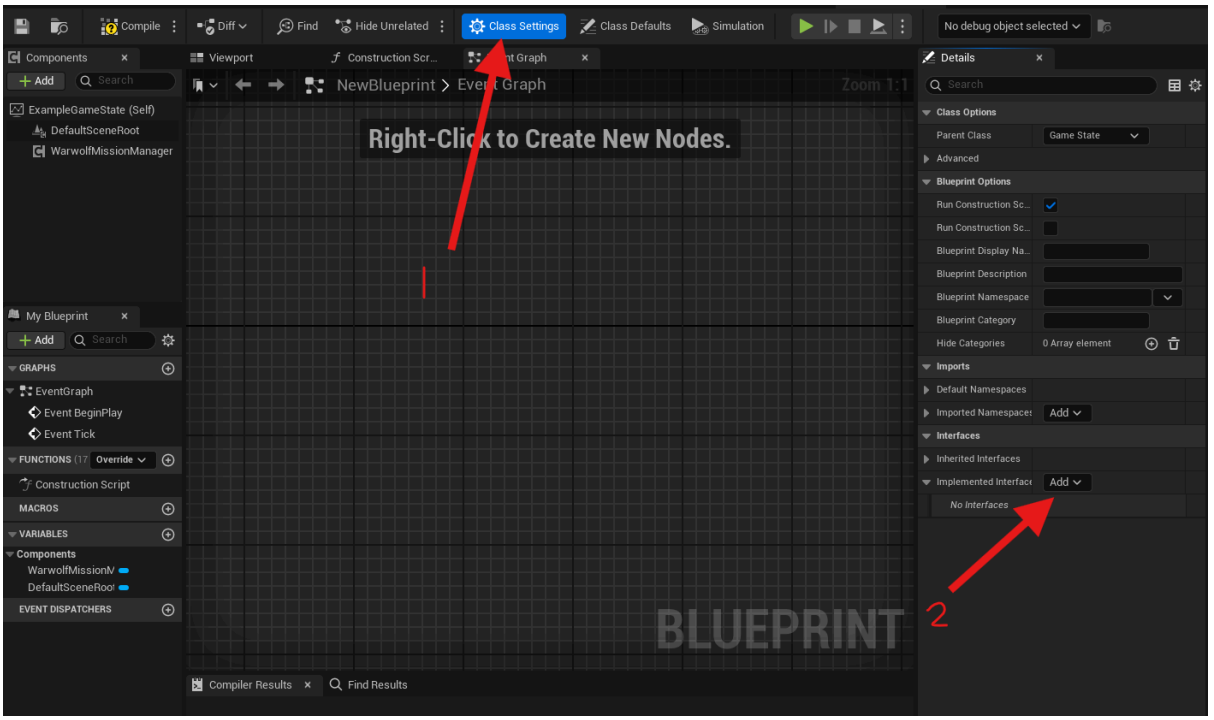
2. Add the Mission Manager component to your game state.

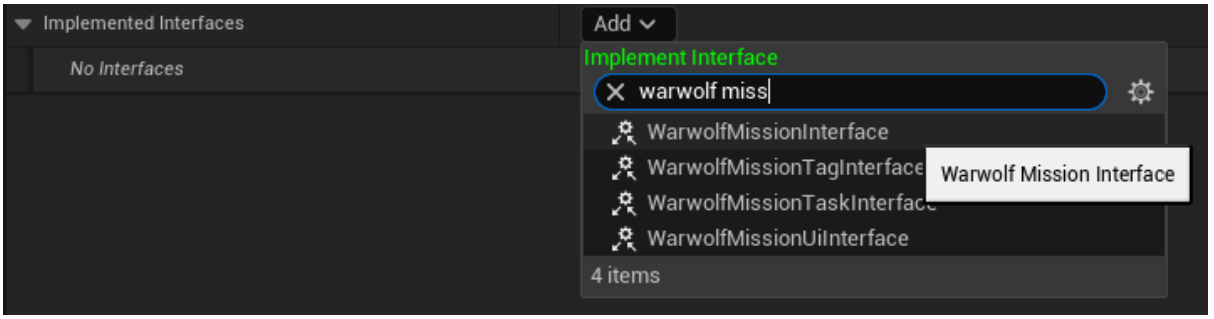


3. Click on the component and bring up the details panel. Untick ‘Start Logic Automatically’

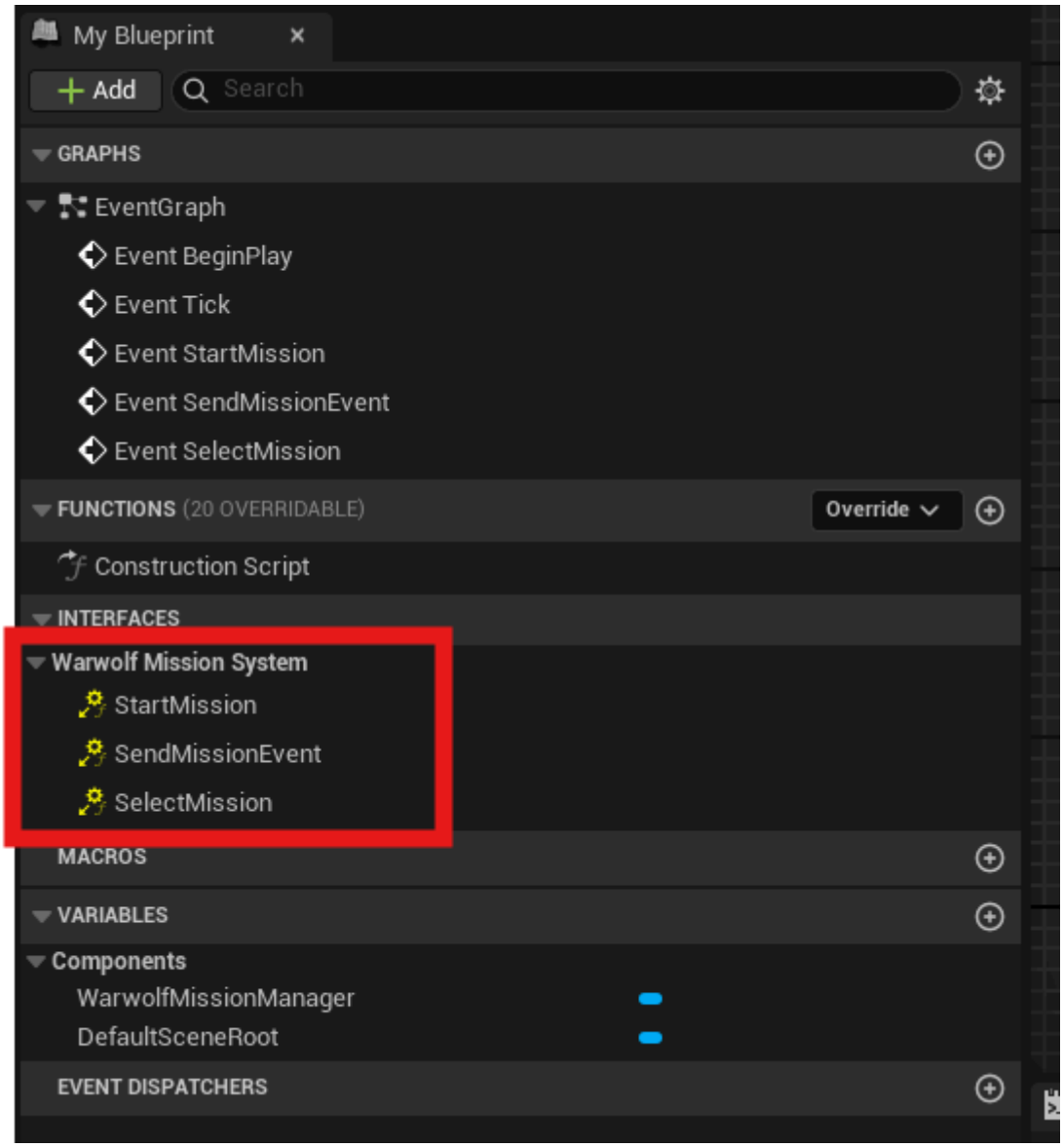


4. Go to the class settings, and add the Warwolf Mission Interface.

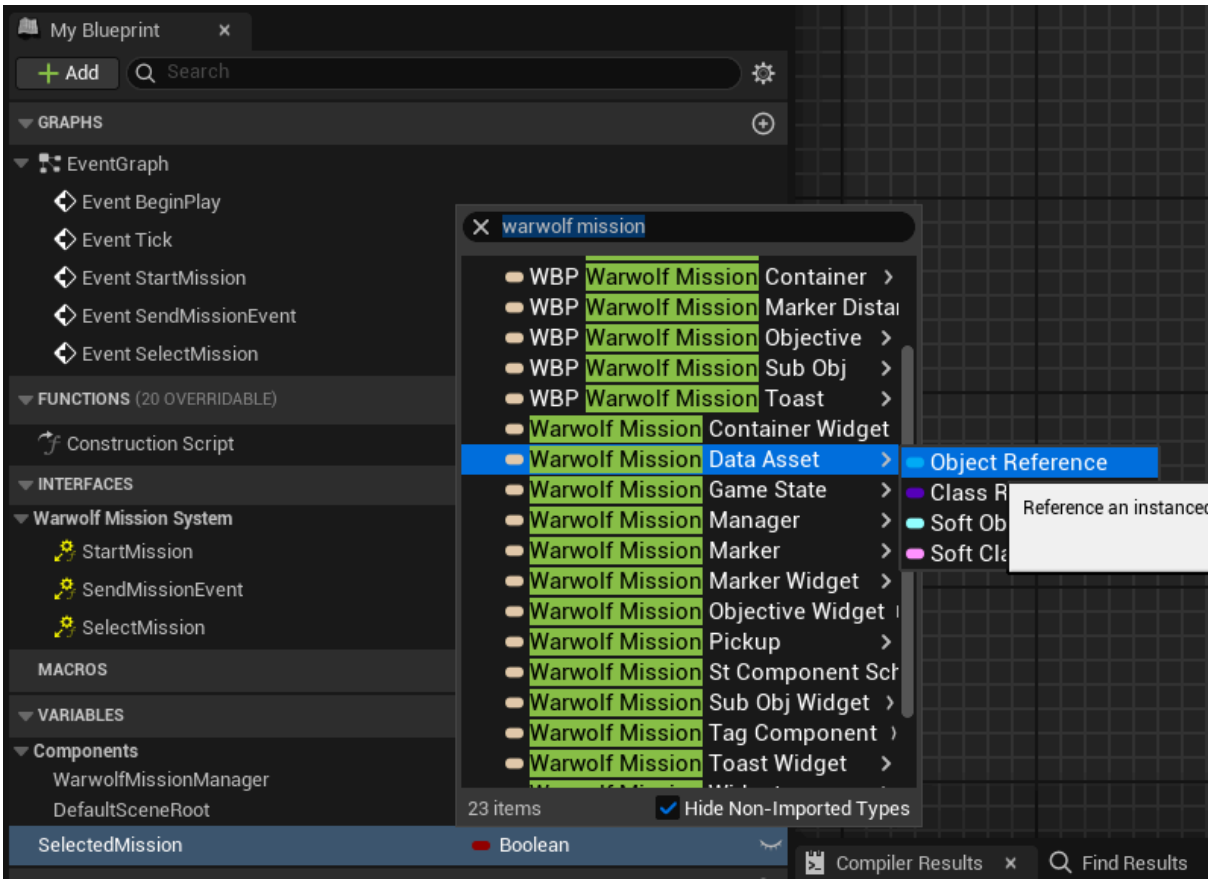




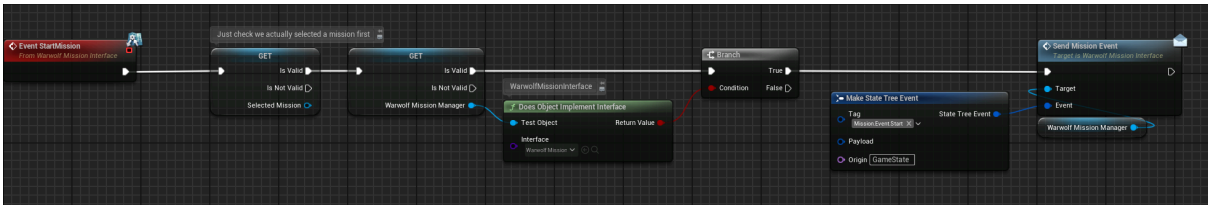
5. Three functions should now have appeared in the ‘My Blueprint’ tab. Double-click all 3 of them so the event nodes appear on the blueprint.

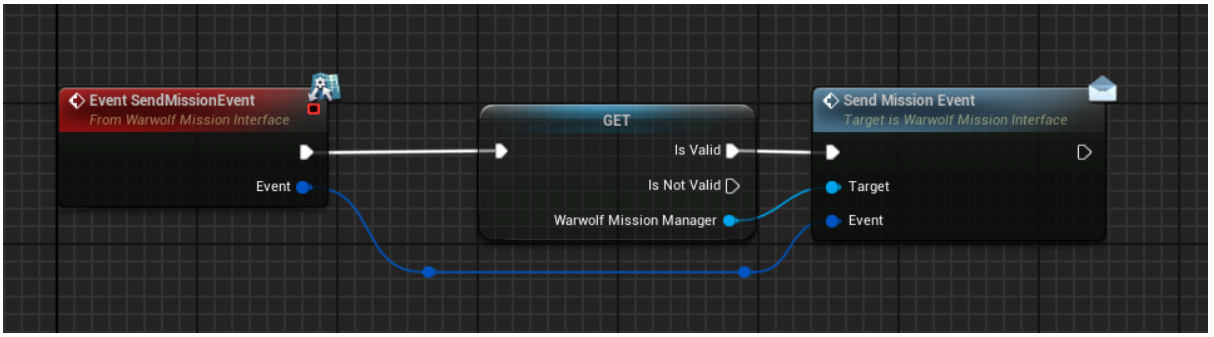


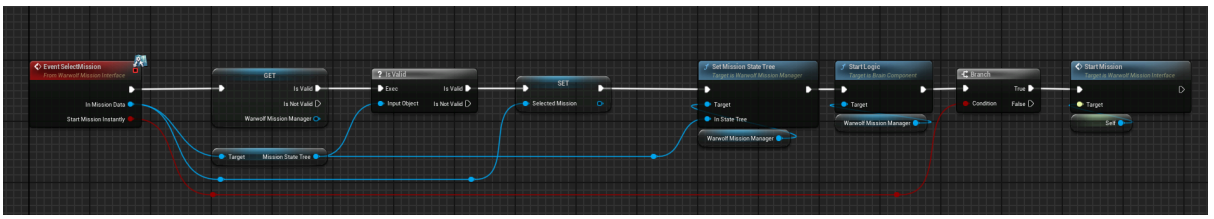
6. Create a new variable called ‘SelectedMission’ and make it a WarwolfMissionDataAsset type.



7. To keep things simple, set up the three functions. If you’re wondering how the ‘Get’ nodes have validity pins, right-click your getter node and press ‘Convert to Validated Get’. This will stop errors. It’s also recommended that you add print strings to the fails for error logging, but again, I’m keeping things quick and simple for now.





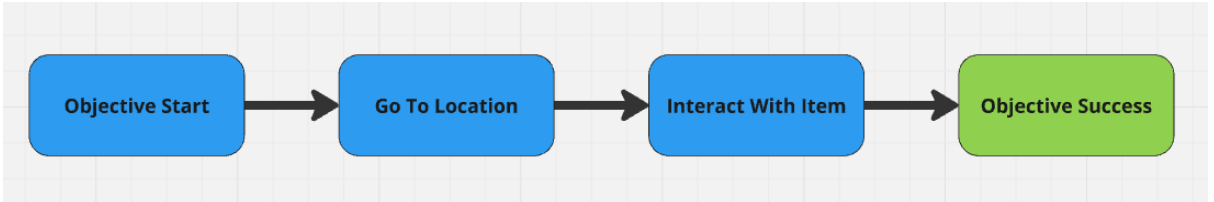


Creating a mission with two simple objectives

We're now going to create a very simple mission with one primary and one optional objective. The aim here isn't to show you how to use all of the tasks but to get to grips with the workflow.

Setting up our primary objective: Reach a location and interact with an object.

1. Let's break down the objective like this:



We are going to need tags to send events to the mission system so it knows when to progress.
Go to the project settings, and then GameplayTags. Add the following tags:
- Mission.Primary.PrimaryExample
- Mission.Primary.PrimaryExample.GoneToLocation
- Mission.Primary.PrimaryExample.Interacted
- Mission.Primary.PrimaryExample.InteractableObject
- Mission.Primary.PrimaryExample.LocationTrigger

- Mission.Primary.PrimaryExample.Marker.FirstLocation
- Mission.Primary.PrimaryExample.Marker.Object

Breaking down each leaf of the tag, we have:
Mission - Just so we know the tag belongs to the mission system

Primary - So we know the tag belongs to some kind of primary objective.

PrimaryExample - What objective it belongs to. This could be anything we want our objective to be known as.

GoneToLocation - We will send this event to the mission system once the player has reached the location we want them to reach.

Interacted - We will send this event to the mission system once the player has interacted with the object we expect them to.

InteractableObject - This is the tag we'll assign to the interactive object. This is so we can activate it at the right time, so players cannot interact with it until we reach the correct state within our objective.

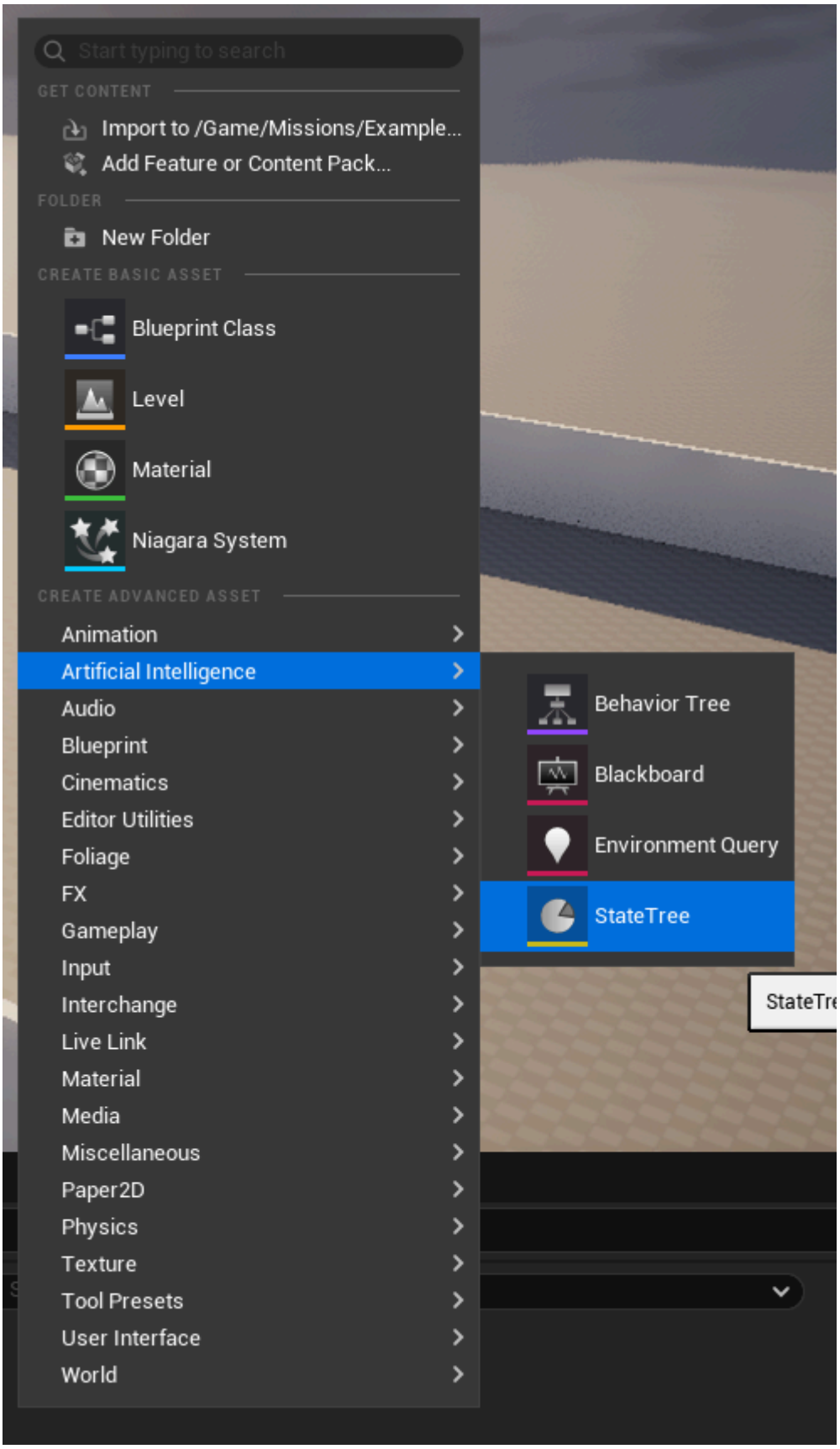
LocationTrigger - This is the tag we'll assign to the trigger box we'll use to let the mission system know we've reached the correct location.

Marker - This is so we know this tag is meant to be applied to a marker as it may share a similar name to another tag.

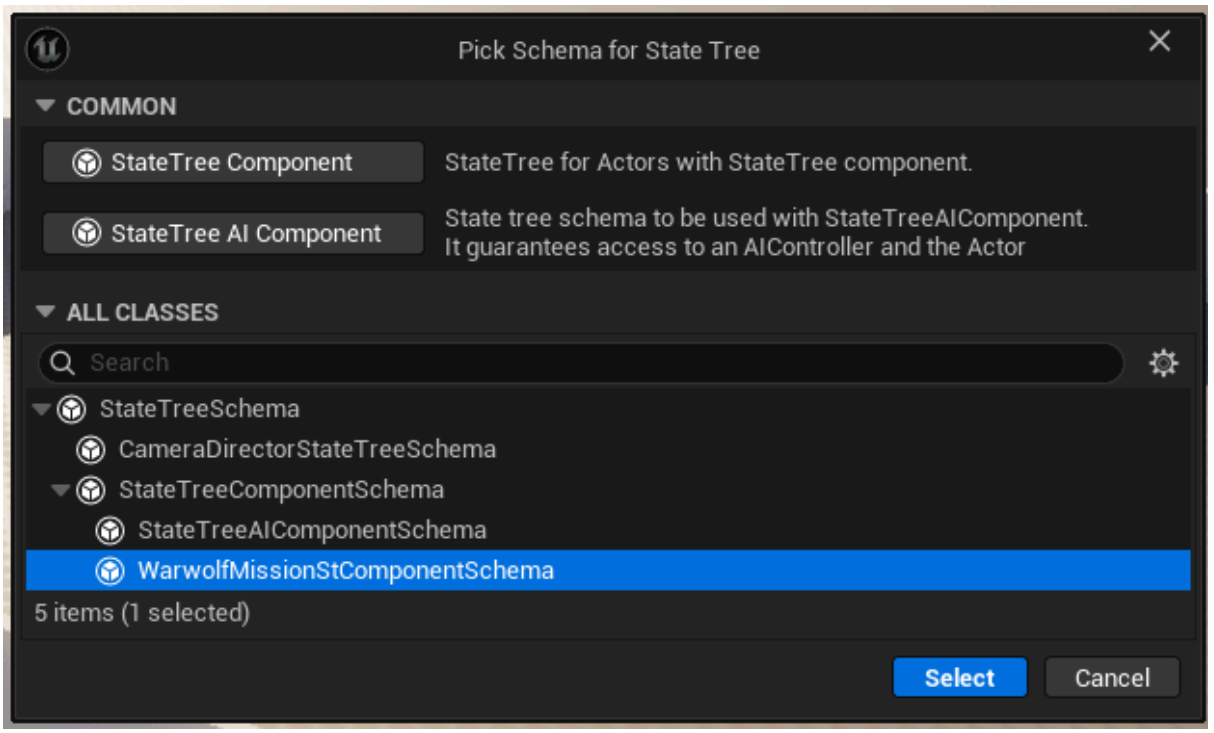
FirstLocation - This is the tag we'll assign to the marker telling the player where to go for the first step of the objective.

Object - This is the tag we'll assign to the marker telling the player where to go when they need to interact with the object.

2. Create a state tree asset in a folder of your choosing. I created a Missions folder and made sub-folders for each mission/objective.

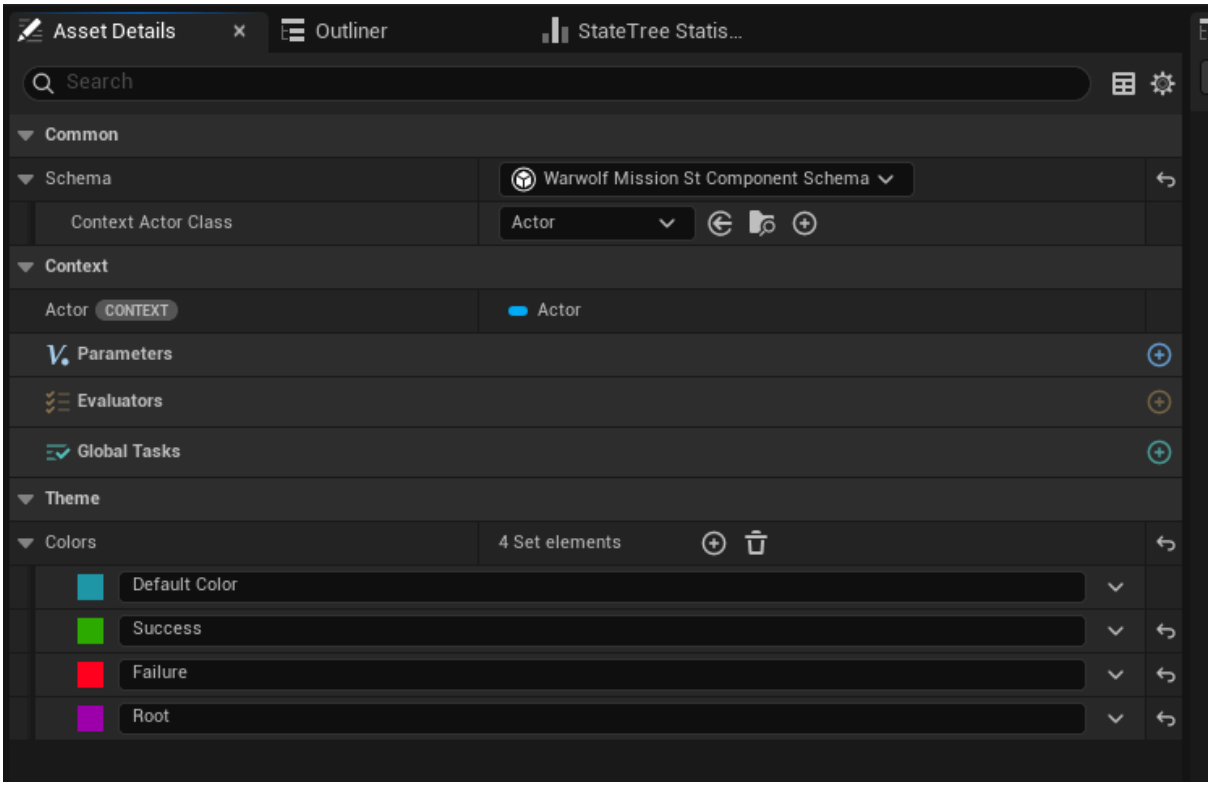


3. You will be asked to pick a schema for the state tree and select **WarwolfMissionStComponentSchema**.

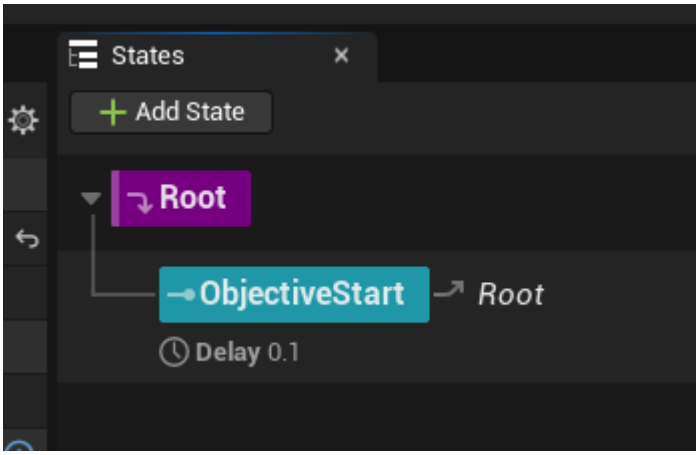


4. Name it whatever you like, but again preferably with **MST_** as a prefix, and **Obj_Pri** or **Obj_Opt** as your suffix, depending on the type of objective it'll be. So in this case, we'll go with MST_Example_Obj_Pri.

5. In the asset details of the state tree, set up 4 colours. Default, Success, Failure and Root. This is just so it's easier to visualise the tree. The colours are totally up to you, but these have worked well for me.



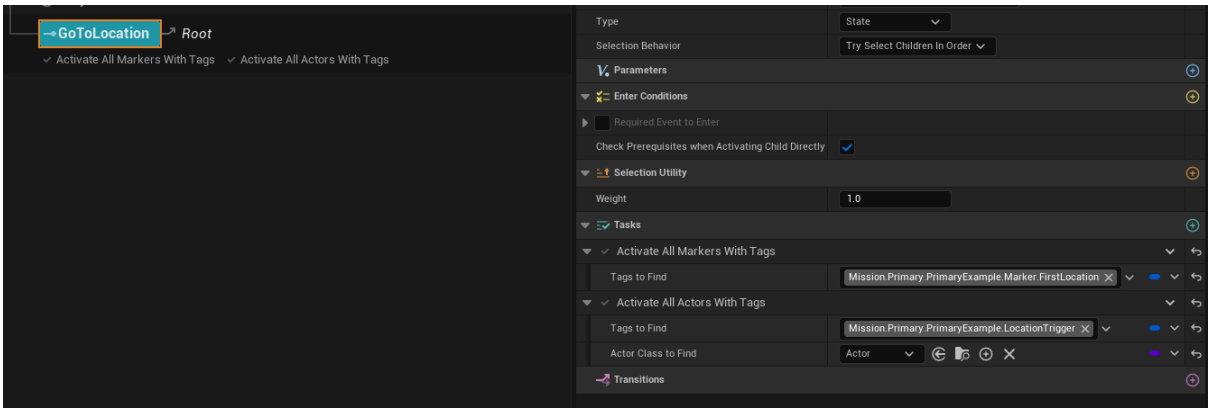
6. Add a new state and call it ‘ObjectiveStart’. Next, add a 0.1 delay to it. We are doing this as good practice, as normally we’d want to add a toast here and delay the state until the toast has disappeared so the objective UI only appears after.



7. Now create another state called ‘GoToLocation’ and add ‘Activate All Markers With Tag’ and ‘Activate All Actors With Tags’ tasks.

   For the markers task, set the tags to find to *Mission.Primary.PrimaryExample.Marker.FirstLocation*. This will find our marker actor and activate it for us.

   For the actors task, set the tags to *Mission.Primary.PrimaryExample.LocationTrigger*. This will find a trigger actor that we will create in the ‘Setting up the level’ section of this tutorial, and activate it.



8. Now we’ll create another state called ‘InteractWithObject’, and similar to the above state, we’ll add the following: Deactivate All Actors with Tags, Deactivate All Markers with Tags, Activate All Actors with Tags and Activate All Markers With Tags.

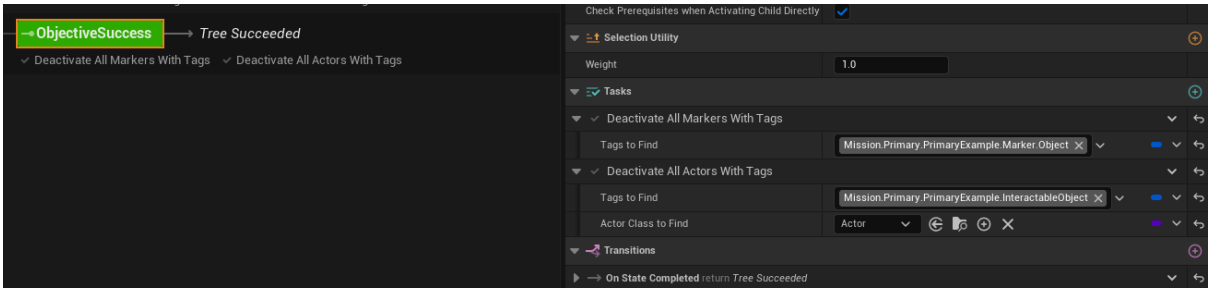
   - For the deactivate actors, we’ll use the tag *Mission.Primary.PrimaryExample.LocationTrigger* and the actor class Actor.
   - For the deactivate markers, we’ll use the tag *Mission.Primary.PrimaryExample.Marker.FirstLocation*.
   This is so we deactivate any markers and actors we used for the previous states.

   - Next, we’ll set the activate all actors Tags To Find to

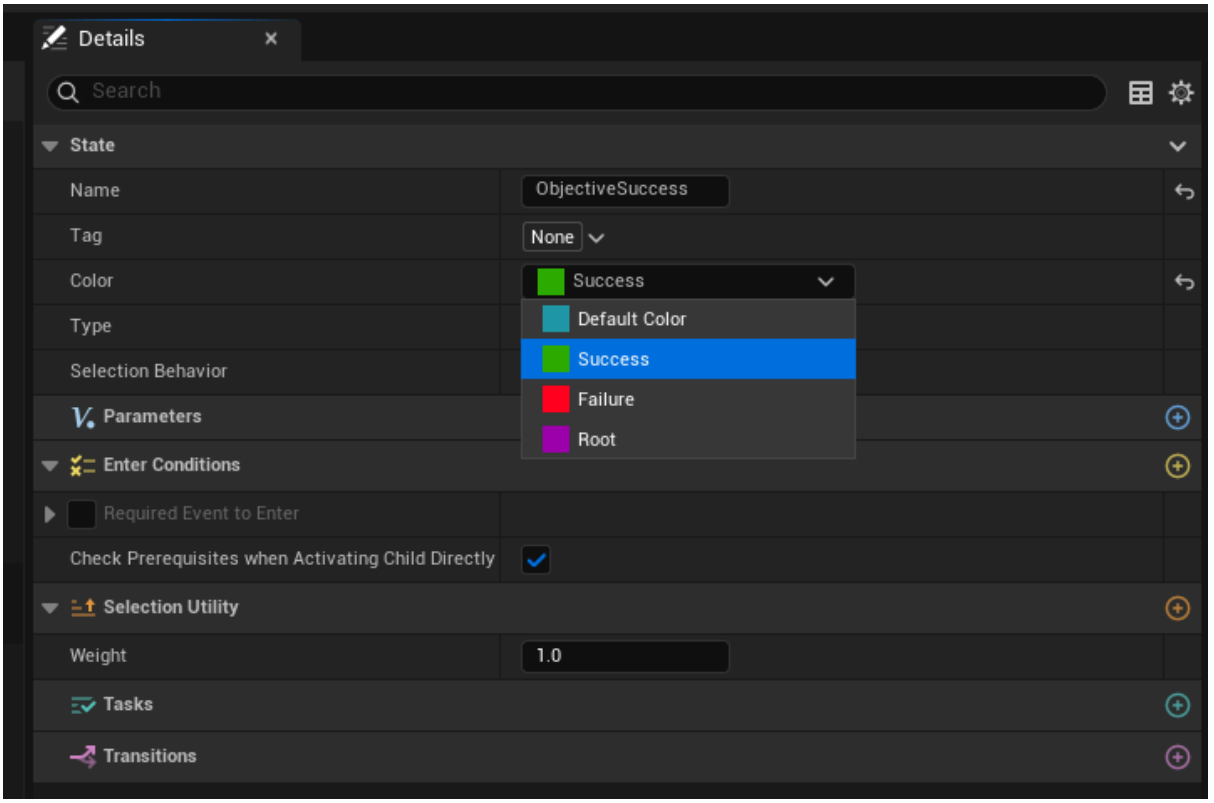
Mission.Primary.PrimaryExample.InteractableObject. We only want to activate this object once the player has reached this point in the objective (though, we can of course not do this if you want to handle edge cases where the player can progress by skipping steps).

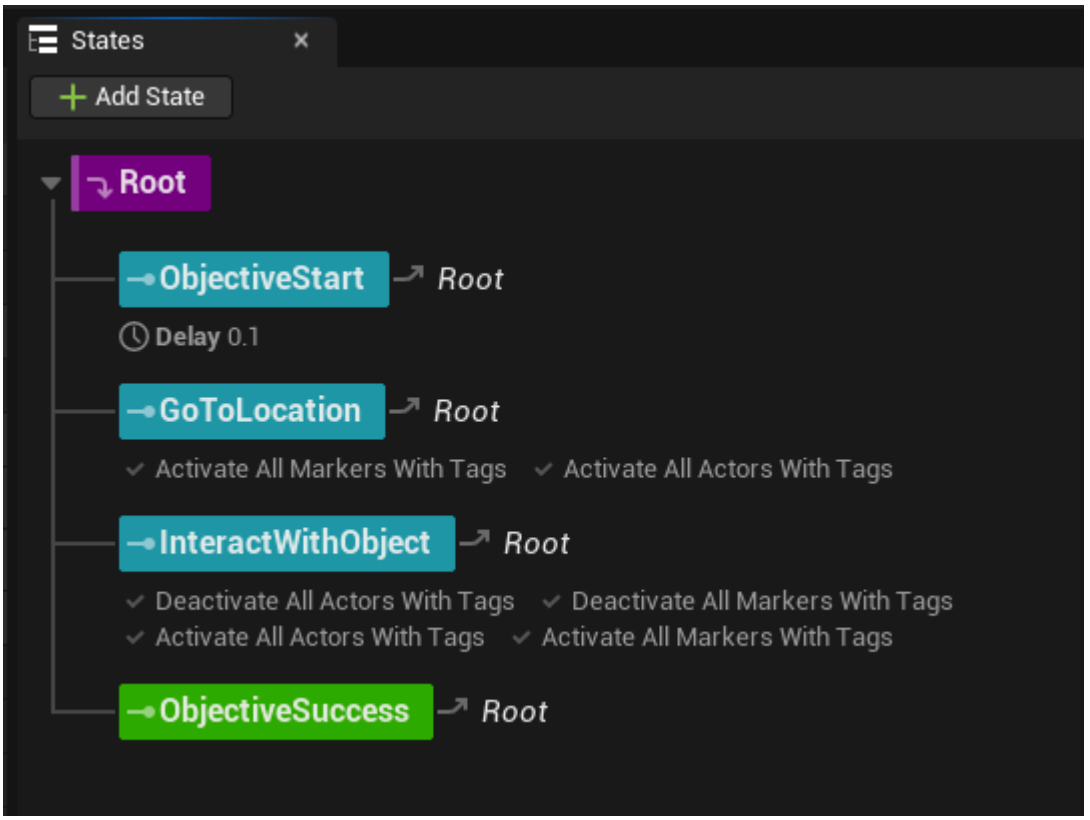
- Finally, we'll set the Activate Markers task's Tags To Find to *Mission.Primary.PrimaryExample.Marker.Object*.

9. We'll now create a new state called 'ObjectiveSuccess'. This is so that we know when the objective is complete, and run any additional completion logic such as awarding XP, etc. For now, let's just deactivate the object marker and actor like so:



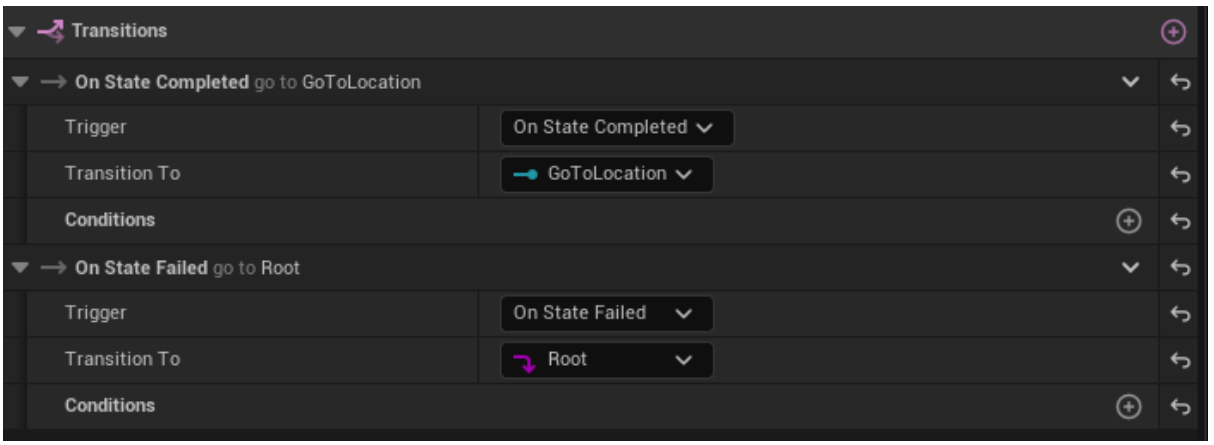
10. Now we're going to quickly set the colours of the states. Set the Root state to our Root colour, and the ObjectiveSuccess state to our Success colour like so:



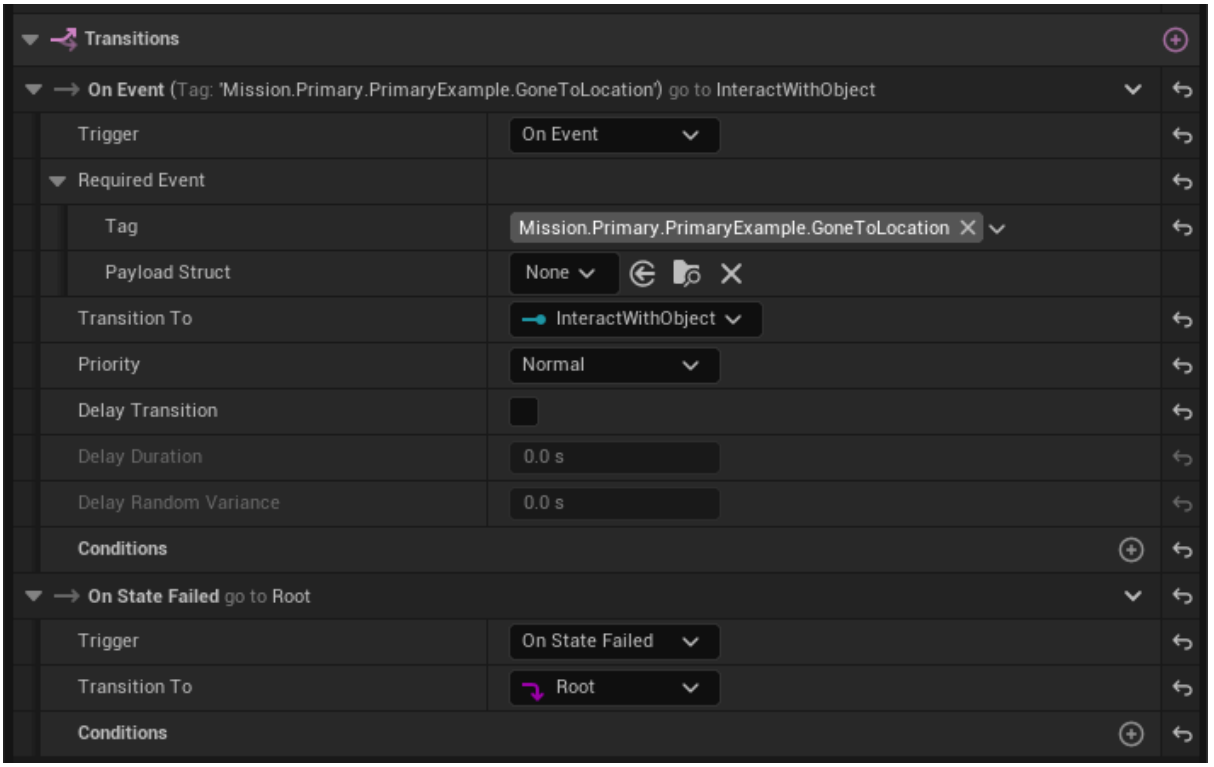


11. Finally, we're going to connect all of our states so that they progress correctly.

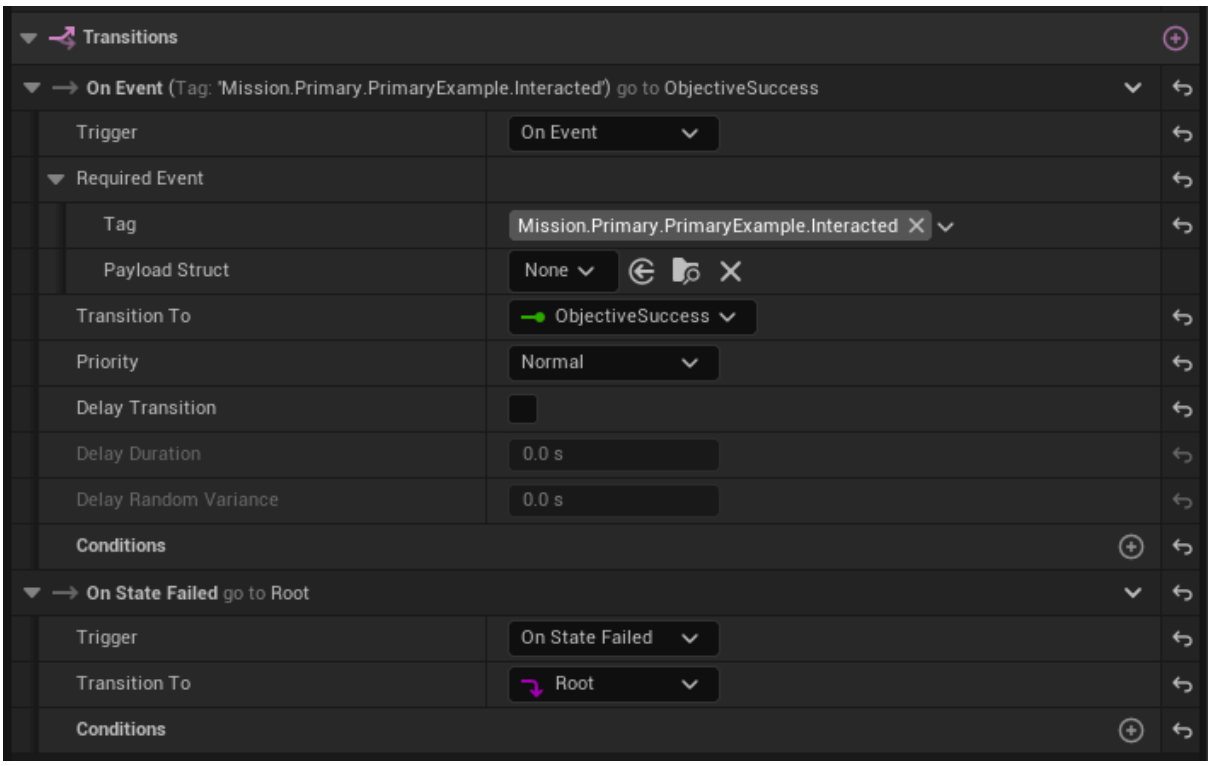
    - In the ObjectiveStart state, we'll set up the transitions like so:



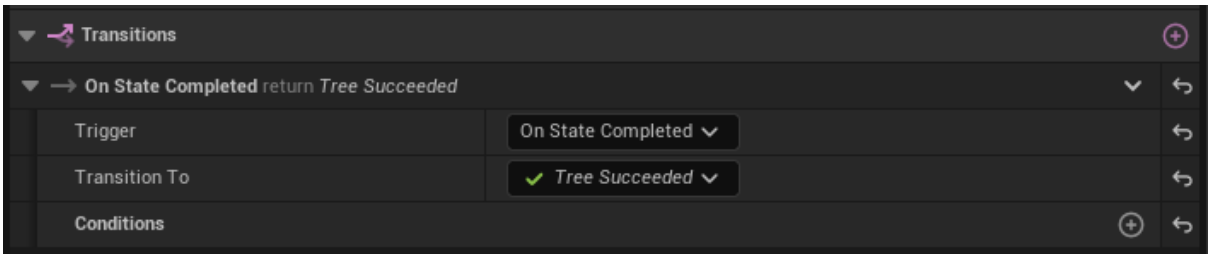
    - In the GoToLocation state, we'll set up the transitions like this:



- In the InteractWithObject state, we'll set up the transitions like this:

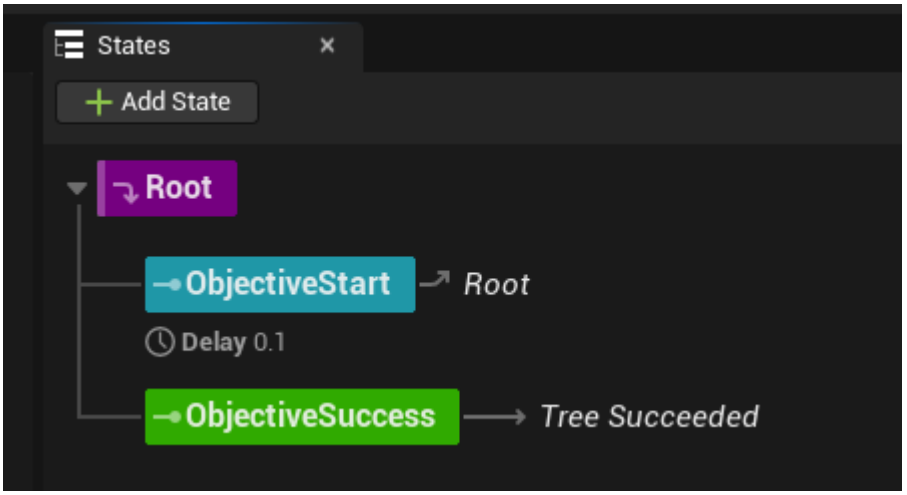


- Lastly, our ObjectiveSuccess state should look like this:



Creating the secondary objective: Reach a location and wait 5 seconds

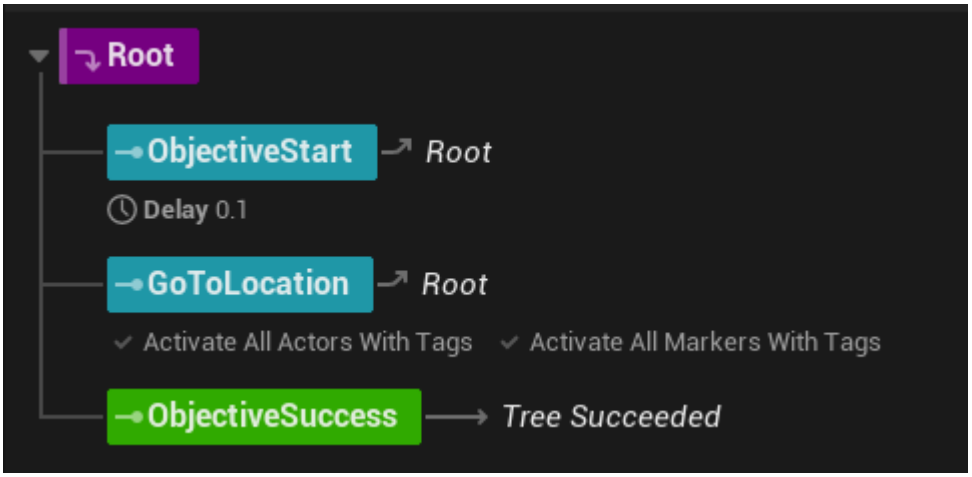
1. To skip repeating a few steps, refer to the above part of the tutorial and create a new state tree with the same schema (if you've been following the exact naming, then use the name MST_Example_Obj_Opt), then set up the state tree colours, the same ObjectiveStart state (with the delay task), and finally your ObjectiveSuccess state. It should look something like this right now:



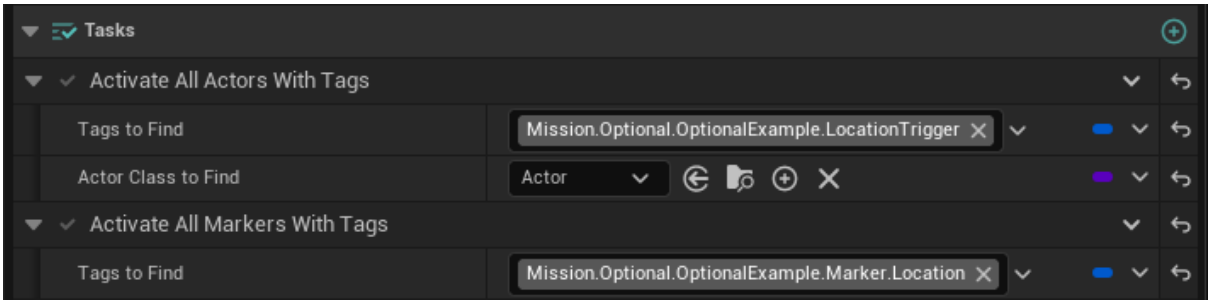
2. Now we're going to set up some more gameplay tags:

   - Mission.Optional.OptionalExample
   - Mission.Optional.OptionalExample.GoneToLocation
   - Mission.Optional.OptionalExample.LocationTrigger
   - Mission.Optional.OptionalExample.Marker.Location
   - Mission.Optional.OptionalExample.Marker.Wait

3. Now, create a new state and move it between the ObjectiveStart and ObjectiveSuccess states like so:

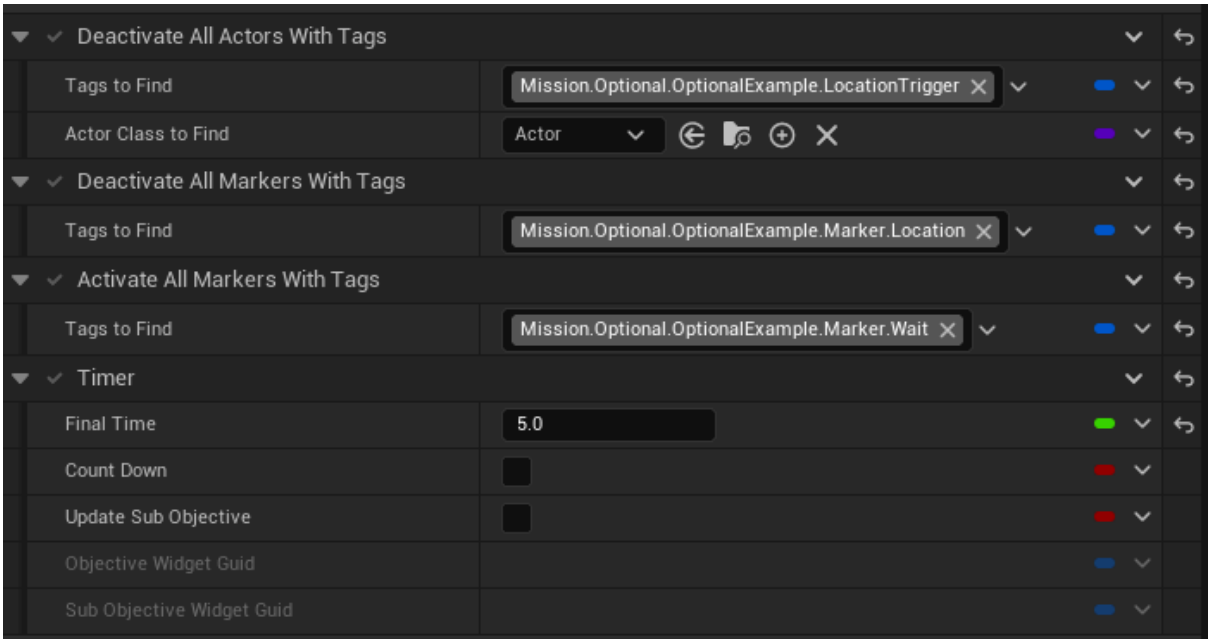


And add the two tasks you can see already on it in the image above. Set them up like so:

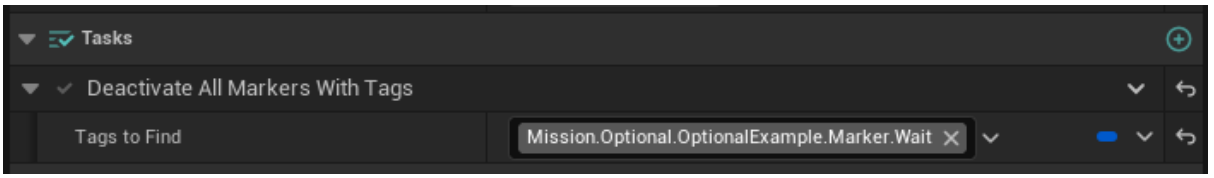


4. Now create another state called Wait5s and move it like so:

   Now add the following tasks to it in this order:

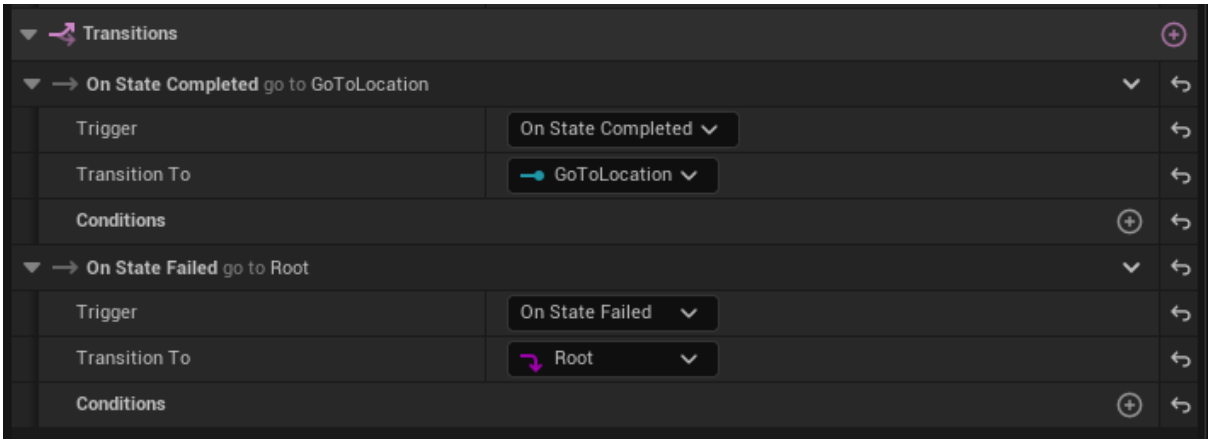


5. Finally, we'll add this to the ObjectiveSuccess state:

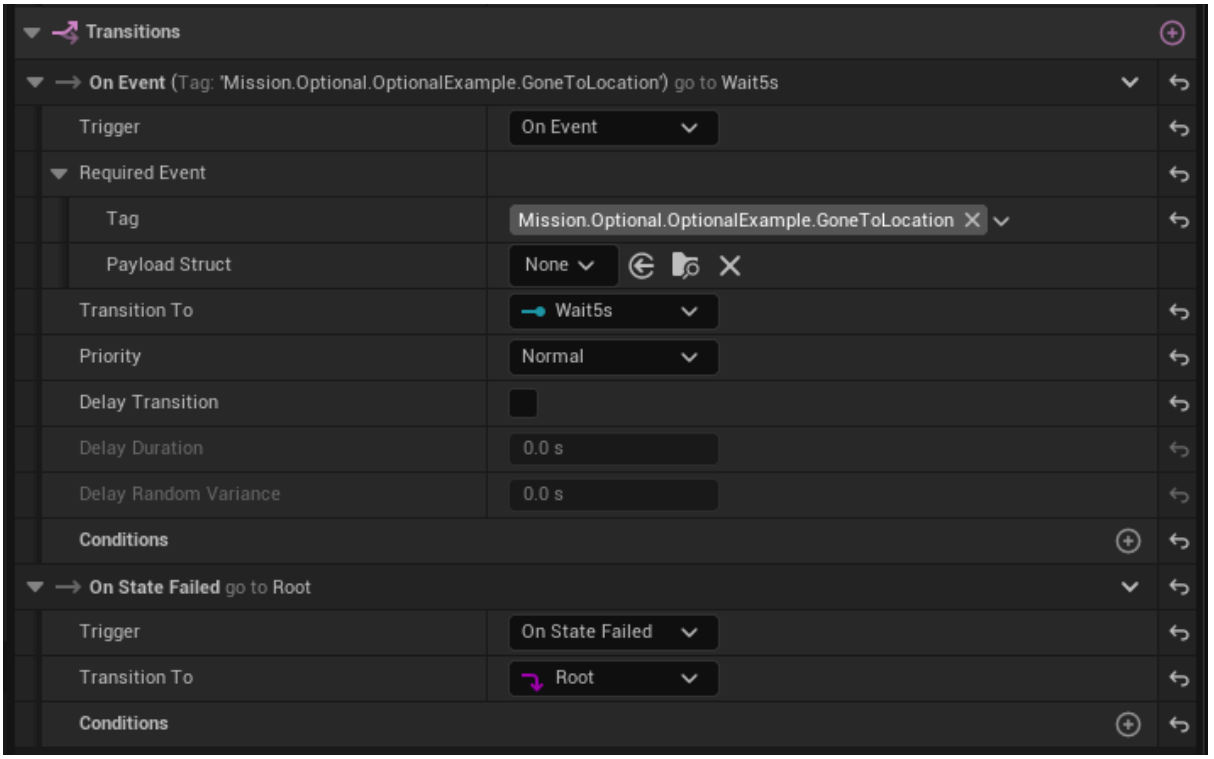


6. Lastly, let's set up our state transitions like so:

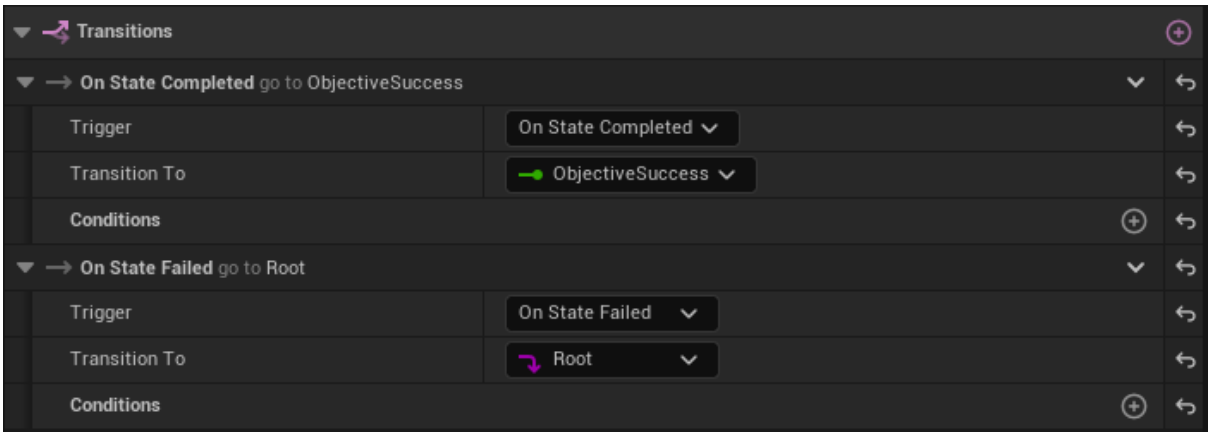
- ObjectiveStart:



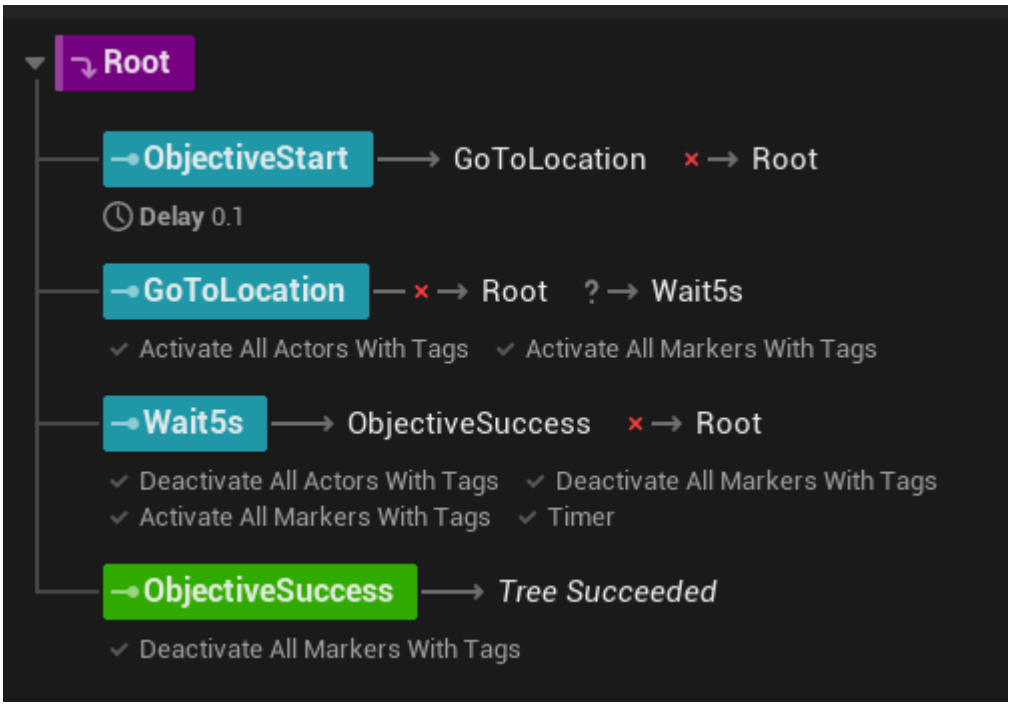
- GoToLocation:



- Wait5s:



The objective should now look something like this:

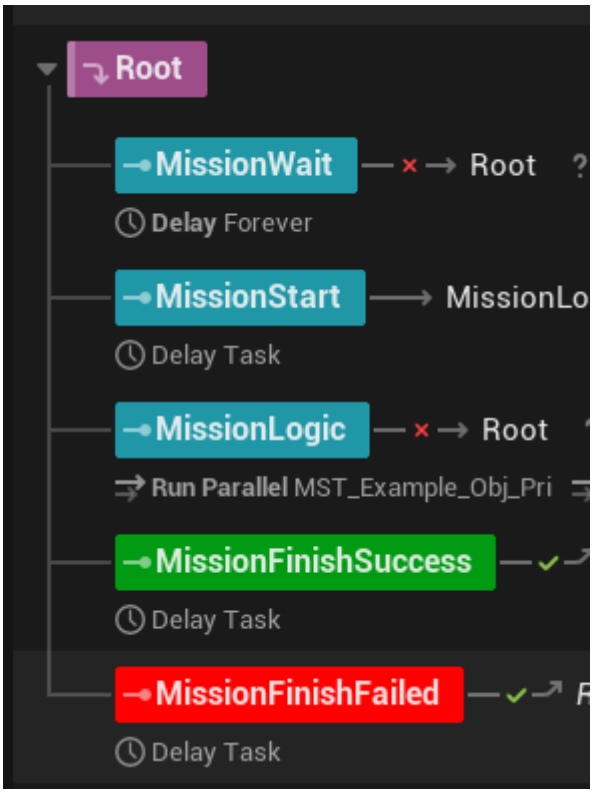


We're now done with the secondary objective. We'll now set up the mission.

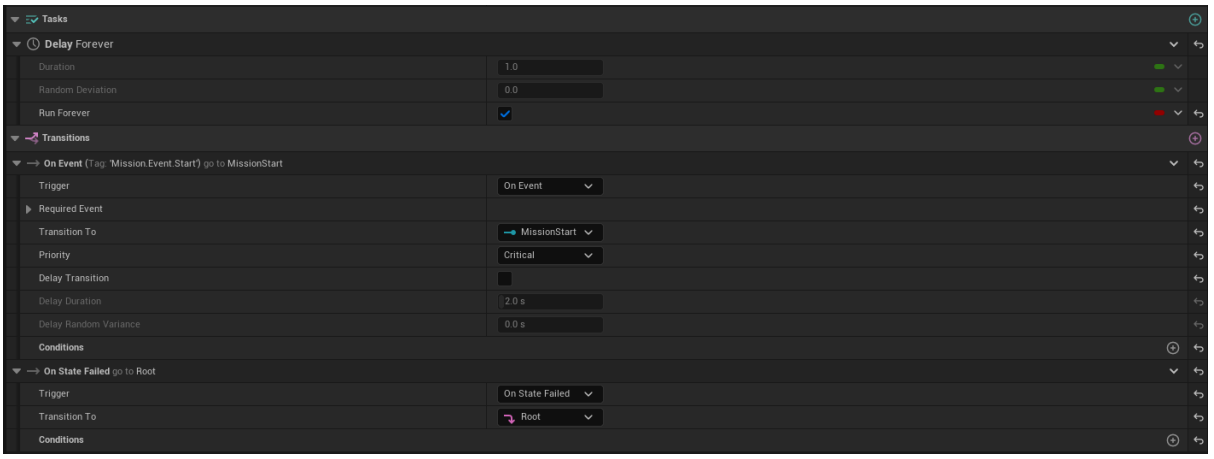
Creating the Mission state tree and data asset

1. Now let's create one more state tree. Similar to before, prefix it with MST_, and suffix it with _Mission. For this example, we'll call it **MST_Example_Mission**.

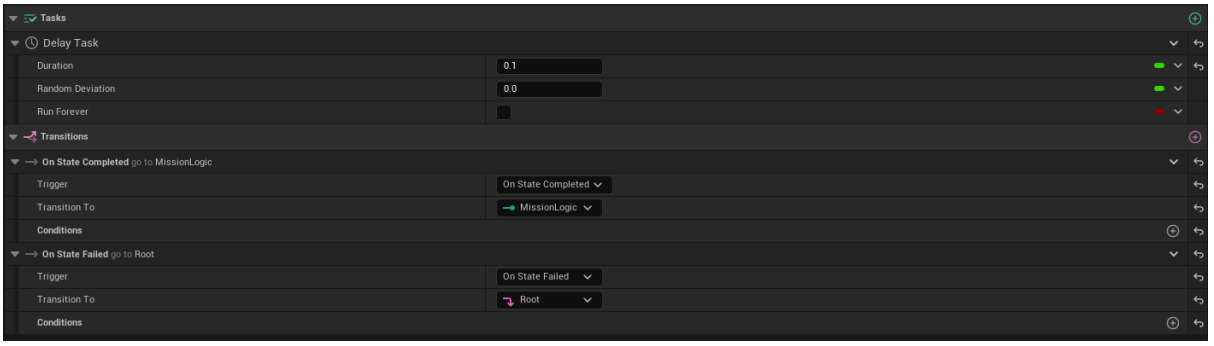
2. Now again, set up these states like so, setting up the colours etc. Ignore the tasks and transitions for now, we'll cover that below.



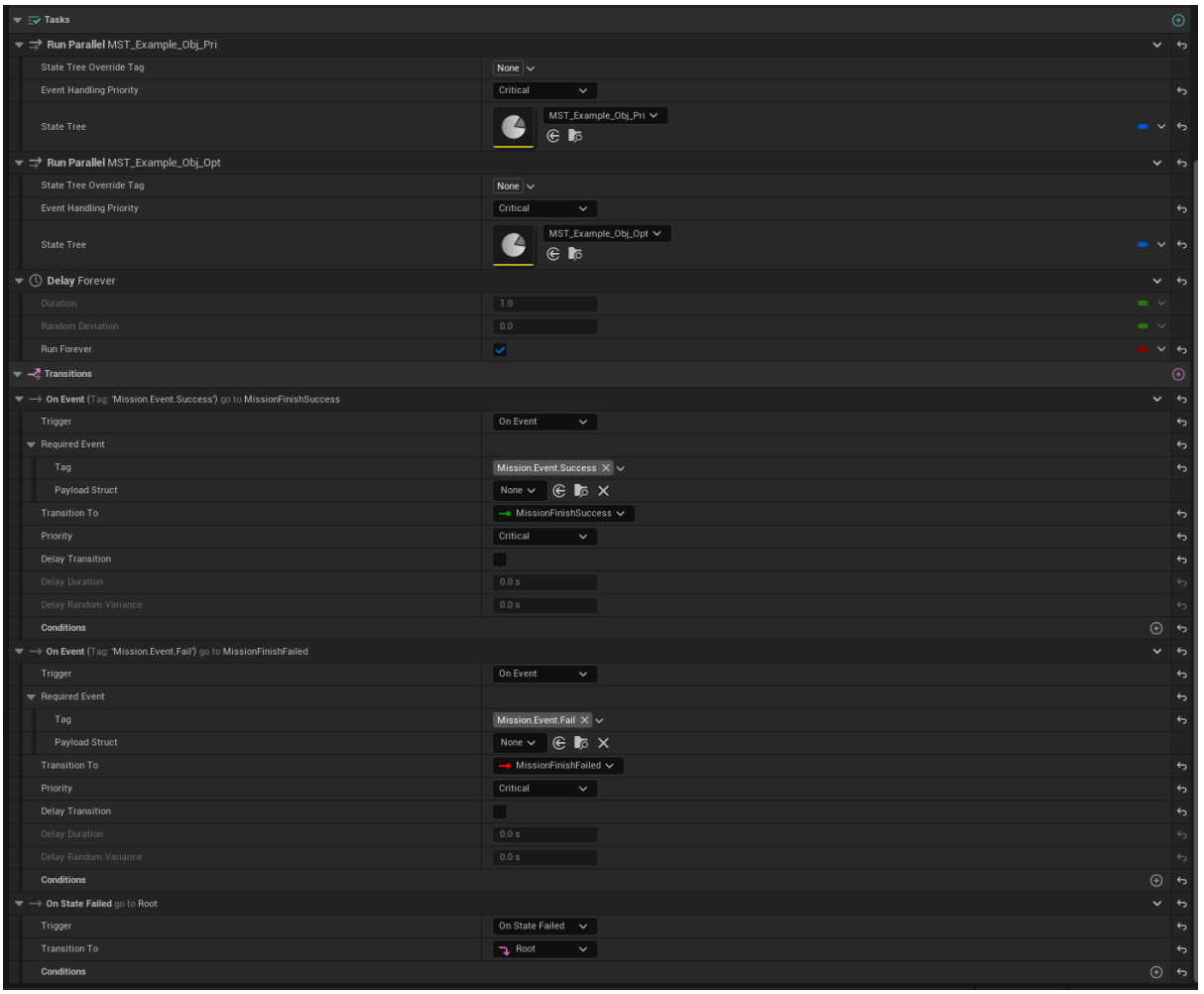
3. For the MissionWait state, use the following tasks and transitions:



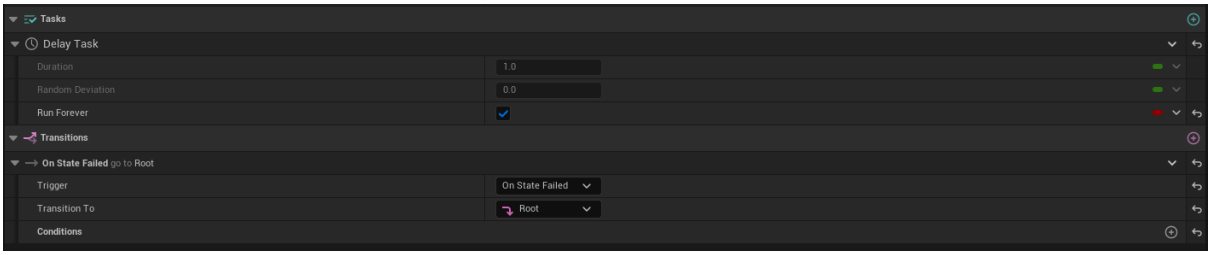
4. For the MissionStart state, set up the following tasks and transitions:



5. For the MissionLogic state, set up the following tasks and transitions:



6. For the MissionFinishSuccess state:

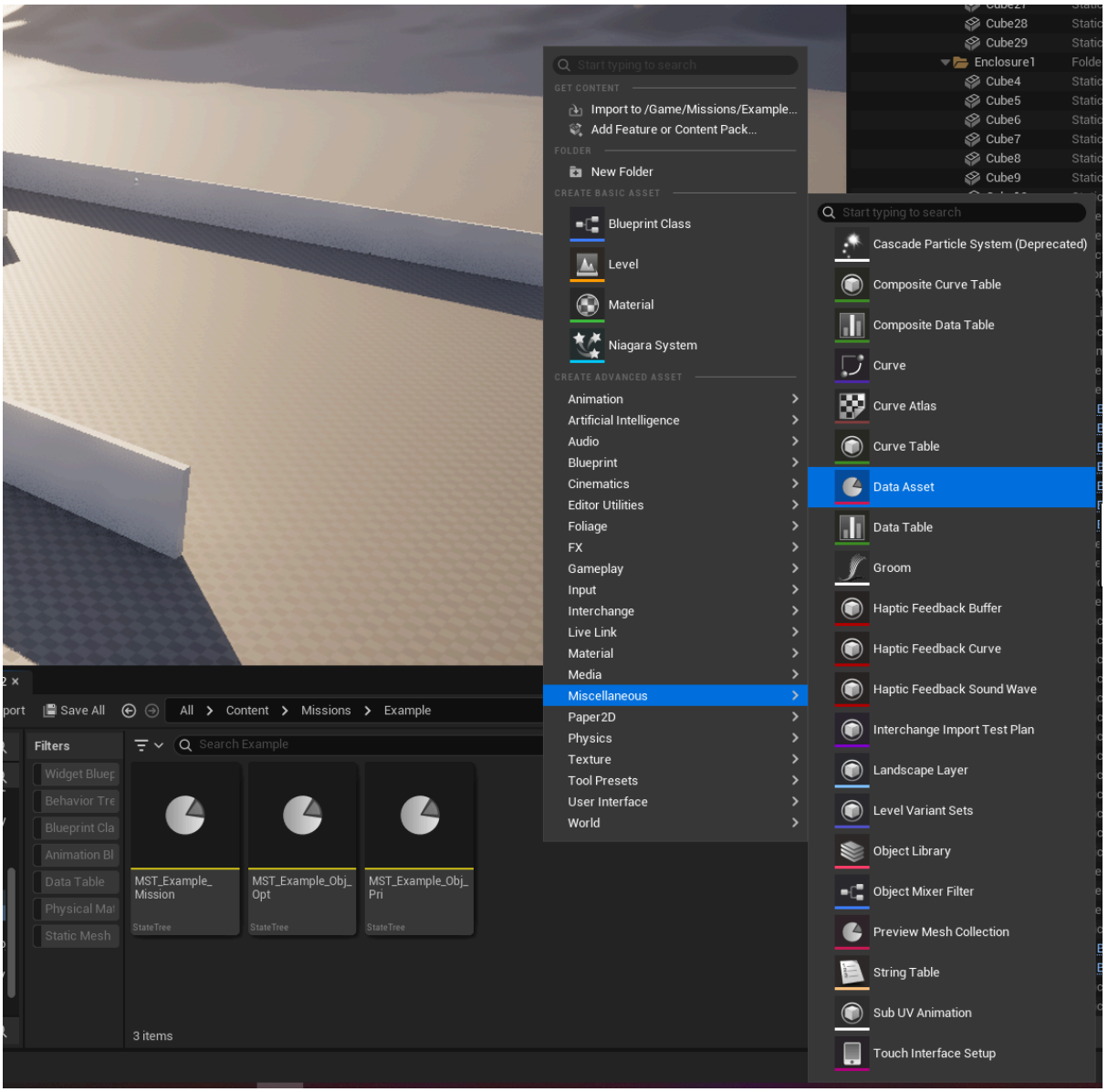


The mission state should now be set up.

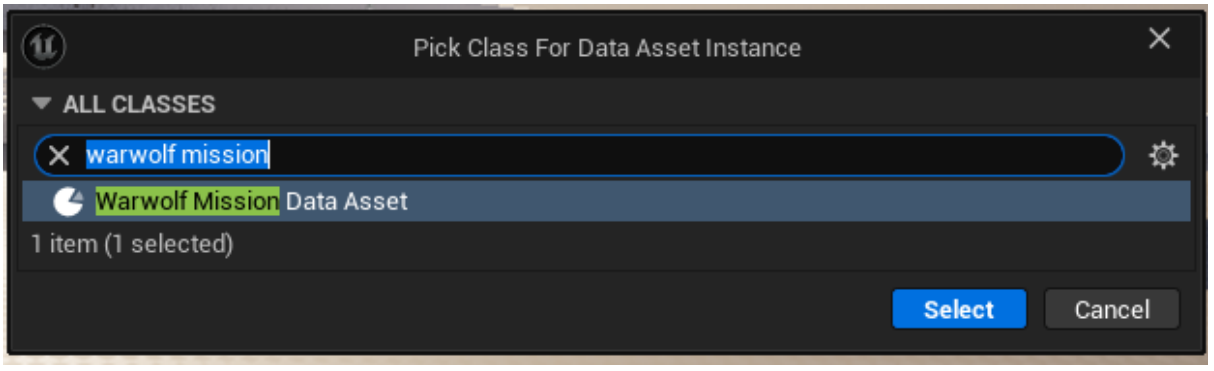
Now we're going to create the mission data asset.

1. We're going to add one more gameplay tag, which is for the mission itself: Mission.Example

2. Right-click in the same folder you've been creating your state trees, and find Data Asset:

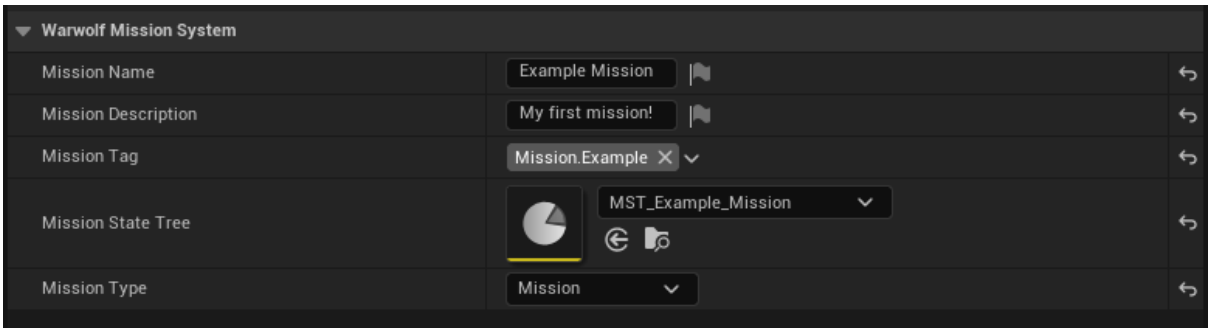


3. When prompted, select Warwolf Mission Data Asset.



4. Name it something like DA_Example_Mission. The DA_ prefix stands for data asset, and the _Mission suffix describes it as a mission data asset.

5. You should set it up like so:



   The Mission Name is what the toast tasks will use (if you've set up your own UI, or used the provided example project's UI).

   The Mission Description isn't explicitly used anywhere in the example, but can be used for mission select screens, or just to add context to designers.

   The Mission Tag is used for logging, and potentially any differentiating similar missions.

   The Mission State Tree should be the mission state tree we just set up, which if you've followed my suggestions would be MST_Example_Mission.

   The mission type should be 'Mission'.

6. The mission is now set up.

Setting up the level

Now let's create a level and place everything we need in it for the mission to successfully complete.

1. Create a new level. I'd just use a basic level.

2. Make sure the game mode and game state are correctly set. We want our mission system to run after all.

3. Now let's create two new blueprints. A trigger that we can activate and deactivate from the mission system, and send events to the system.

   a. Create a new actor blueprint and name it BP_MissionTrigger

b. Add a Warwolf Mission Tag component and box collision component:

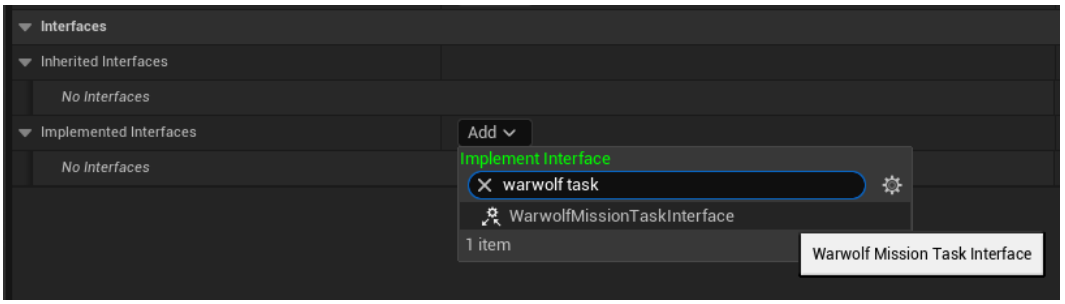




The tag component is how the tasks know what actor has what tags when we use tasks like ‘Activate All Actors With Tag’.

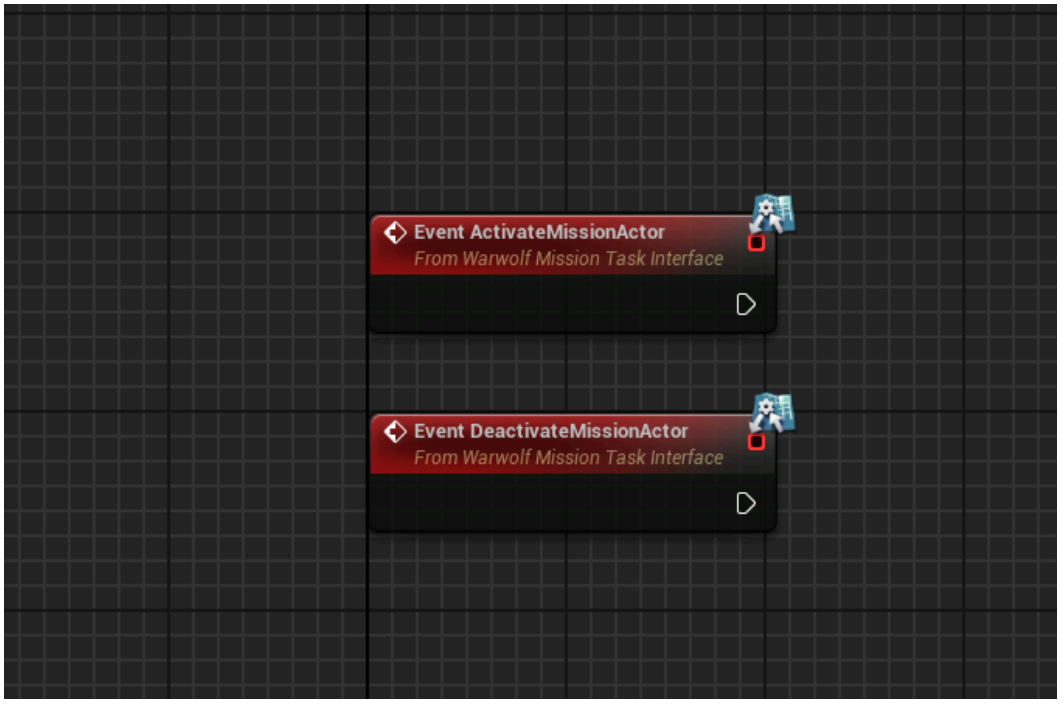
The box collision is naturally so we can listen for overlap events from the player.

c. Add the Warwolf Mission Task Interface:

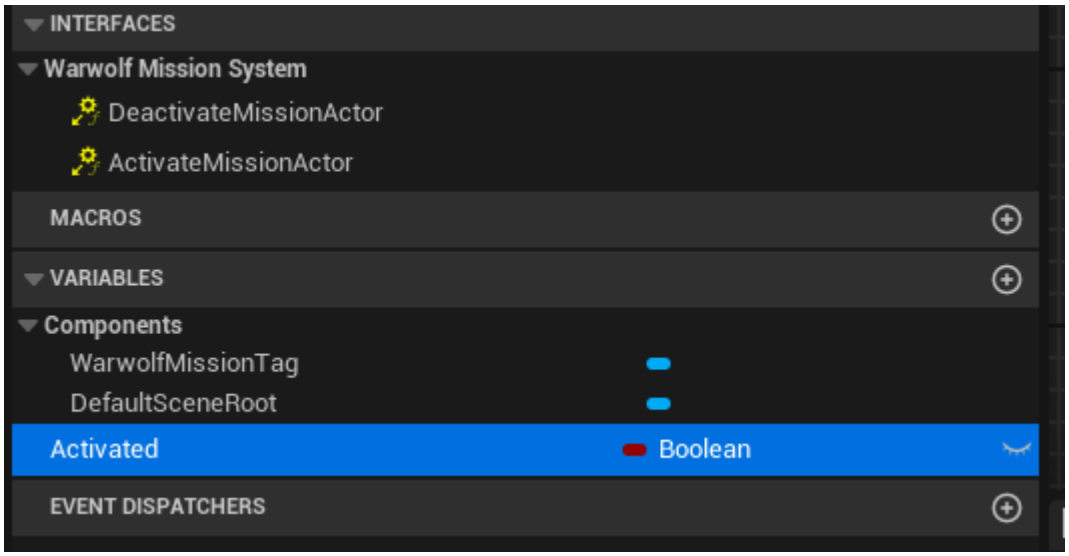


This is how tasks such as ‘Activate All Actors With Tag’ tasks can activate and deactivate actors without specifically needing to access them as a class.

d. Double-click both of the new interface events to get them to appear on the blueprint:



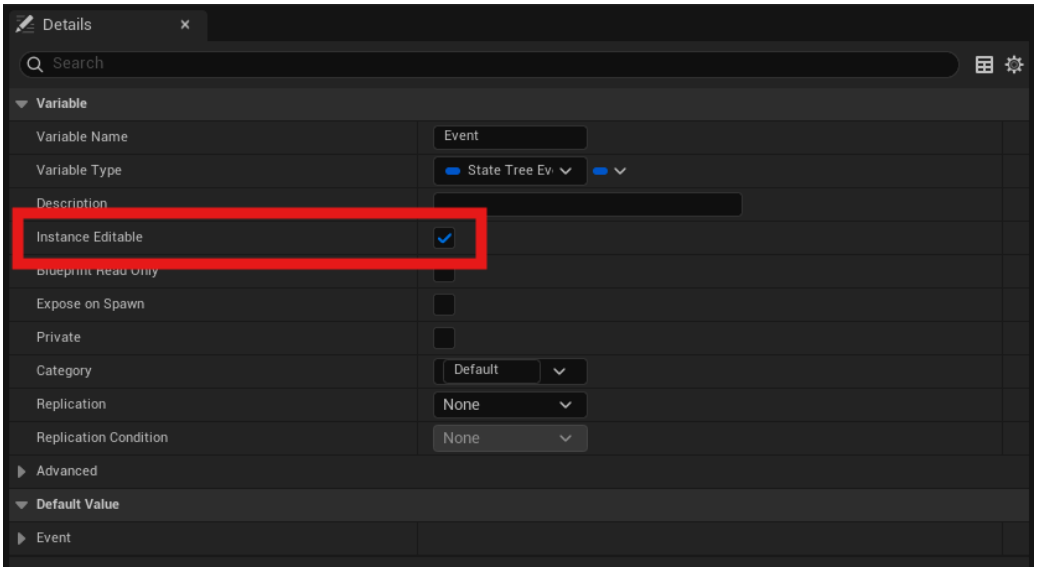
e. Create a new boolean variable called ‘Activated’ it should be false by default:



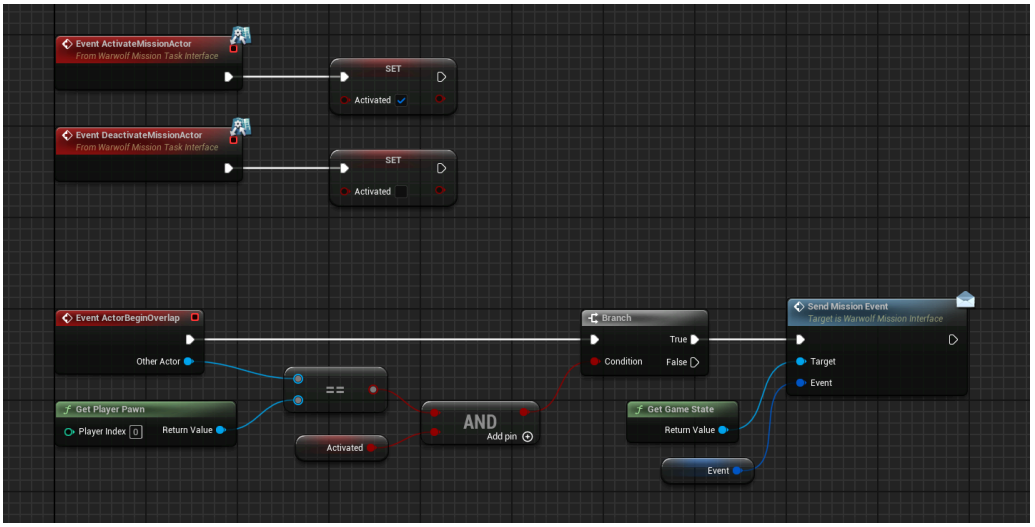
f. Now create a State Tree Event variable called Event.



g. Look at the properties of both of these variables and make sure ‘Instance Editable’ is ticked for both.

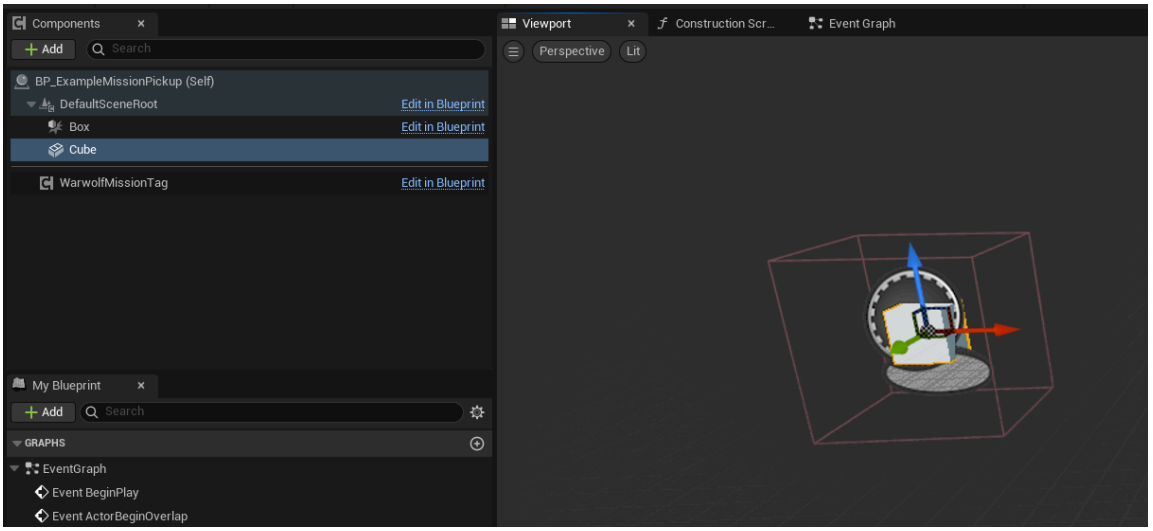


h. Now set up the following blueprint:



We are just locally checking to see if the trigger is activated or not, and if it is, we send a mission event if the player overlaps with it.

4. Now create a child of this blueprint class and name it BP_MissionPickup. It has all of the logic we require, we're just also going to add a cube to it so the player can see it like so:



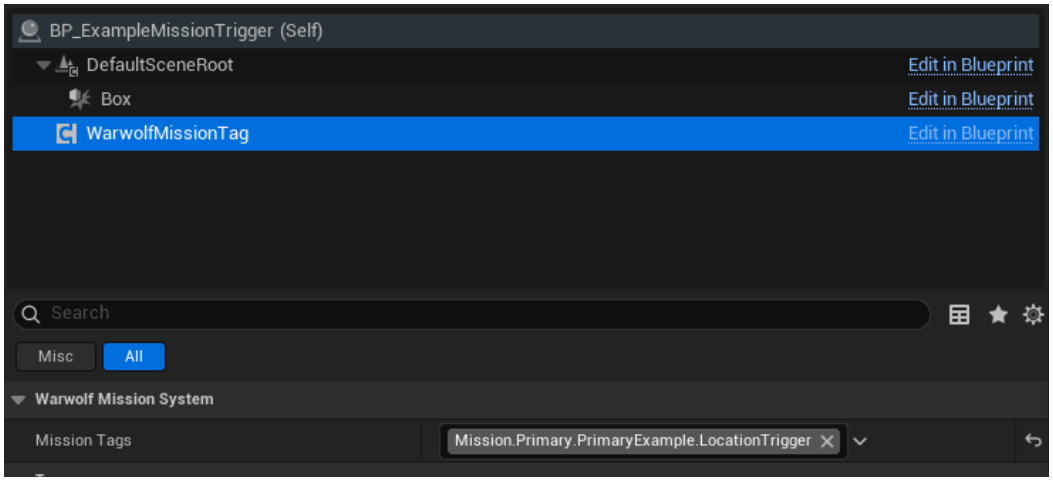
Yep. That's all for now.

5. Now let's place some actors in the level for the mission.

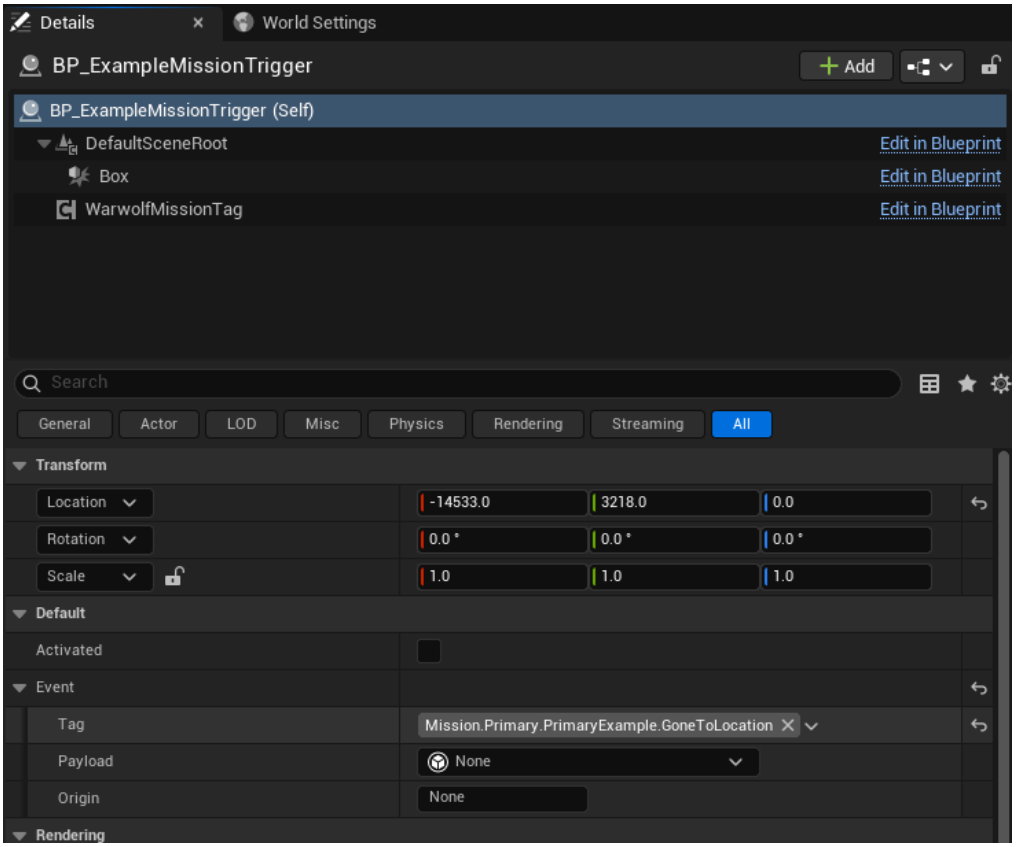
   For the primary mission:

   a. Place one the triggers we just created on one side of the map. Make the box collision cover the area we want the player to enter.

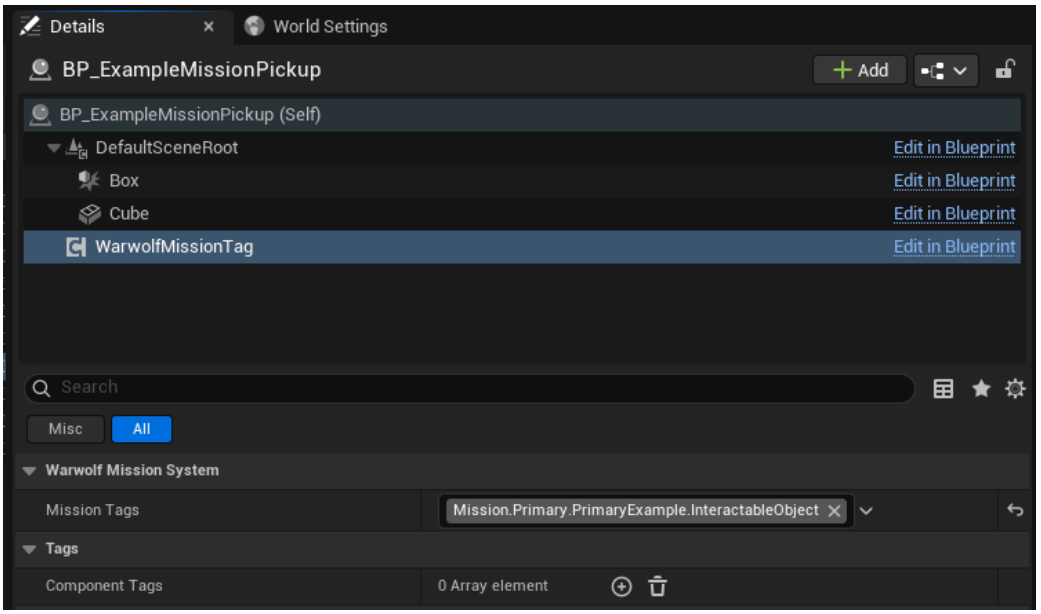
   b. Select the tag component on the trigger and add the following tag:



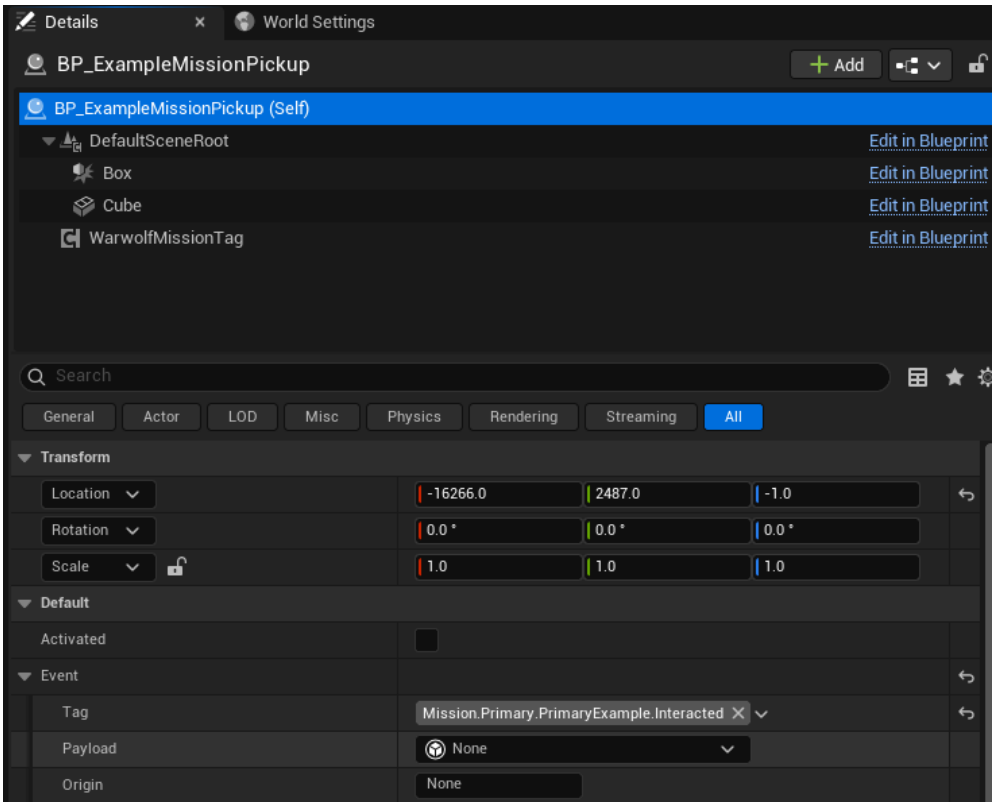
   c. In the trigger's properties, add the following tag to the Event variable we created:



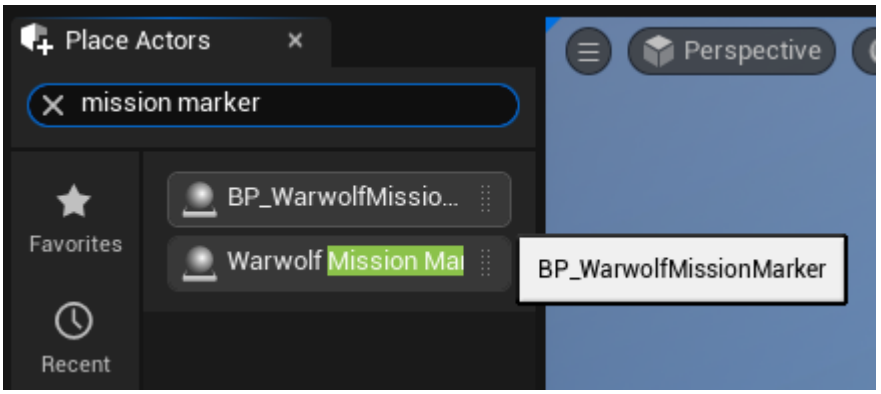
d. Now let's add the pickup within that area. Place a BP_MissionPickup within the box collision somewhere, anywhere. Click on the tag component and add the following tag:



e. Now in the pickup's properties, add the following tag to the Event property:

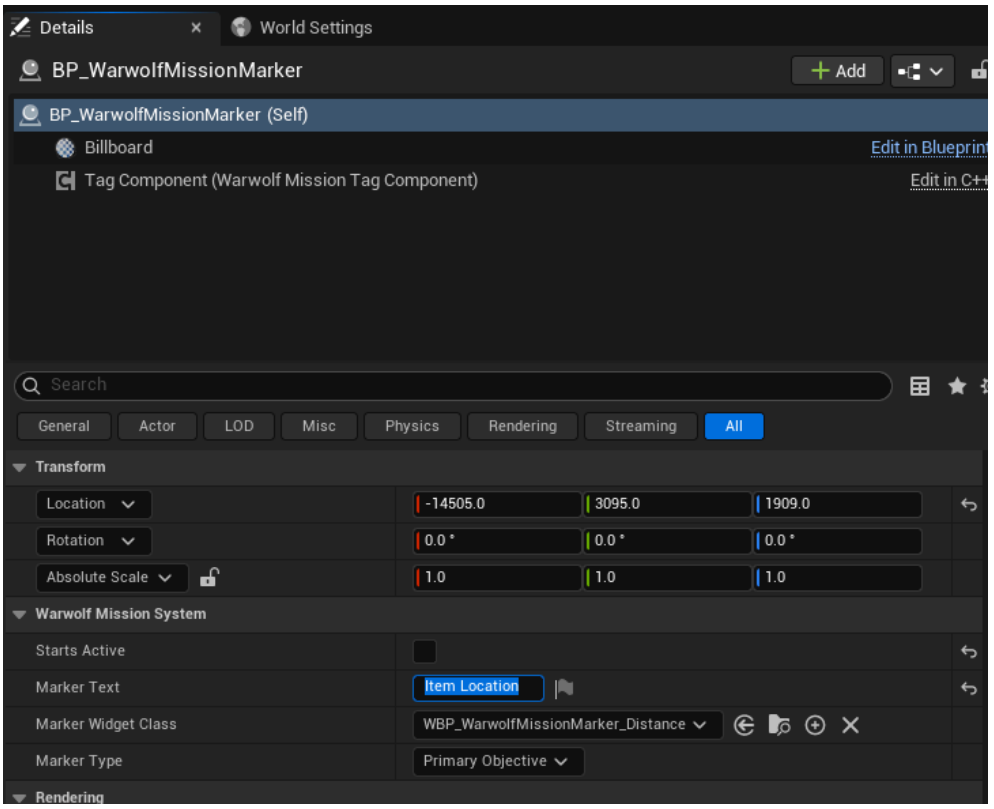


f. Finally, let's add two markers. In the Place Actors tab, search for 'mission marker' and select BP_WarwolfMissionMarker.

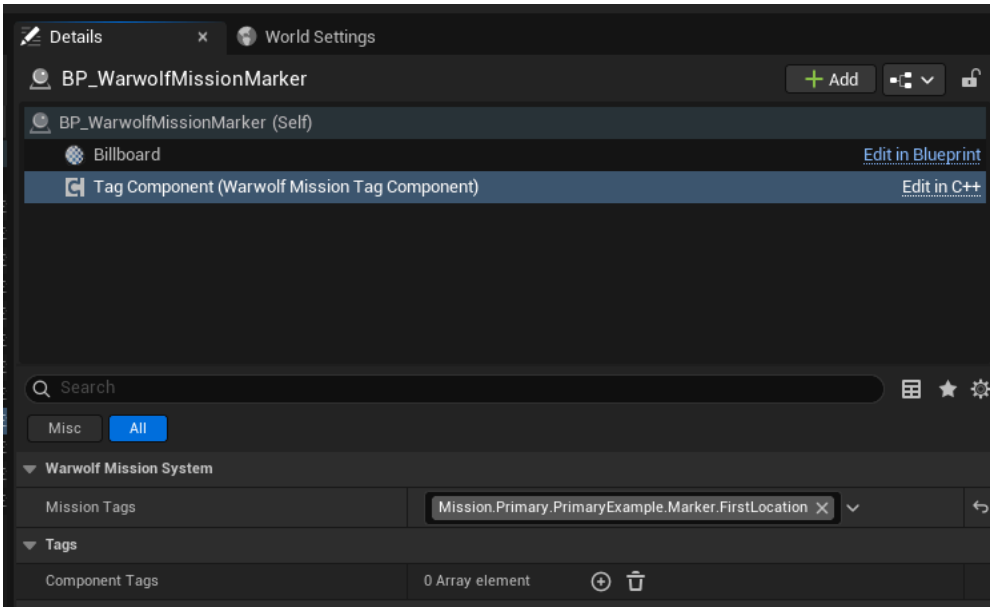


Place two of them down. Move one to the centre of your trigger location and one above your interactable object.

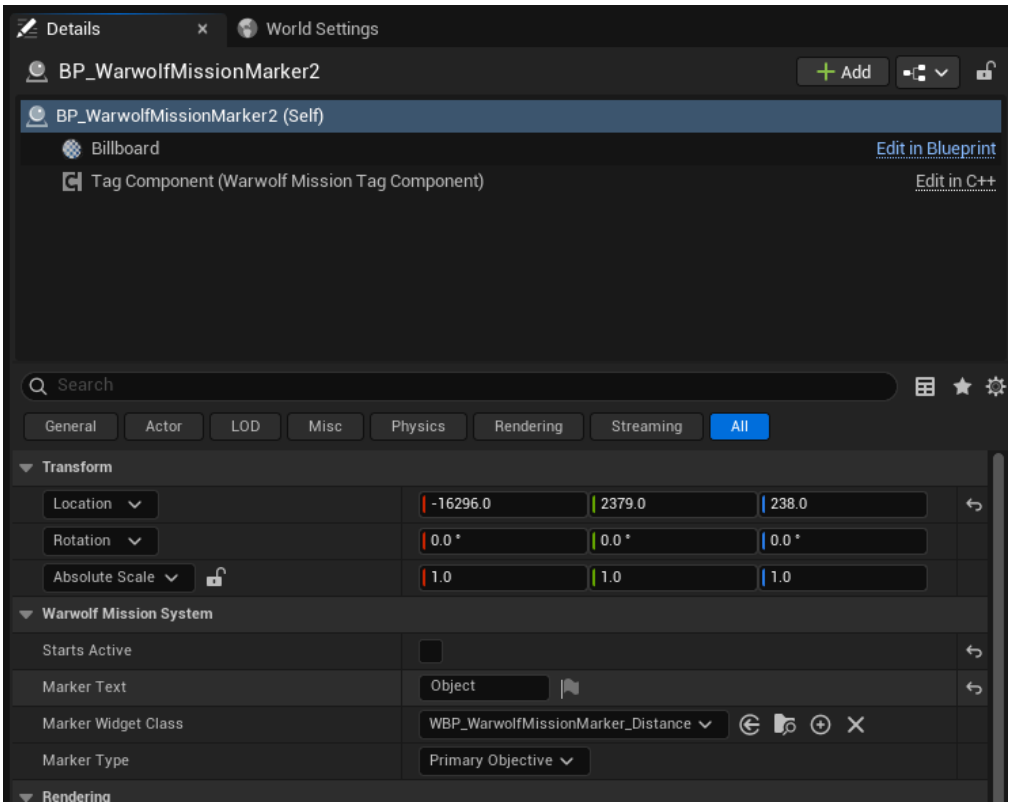
g. In the location's marker, set the following properties:

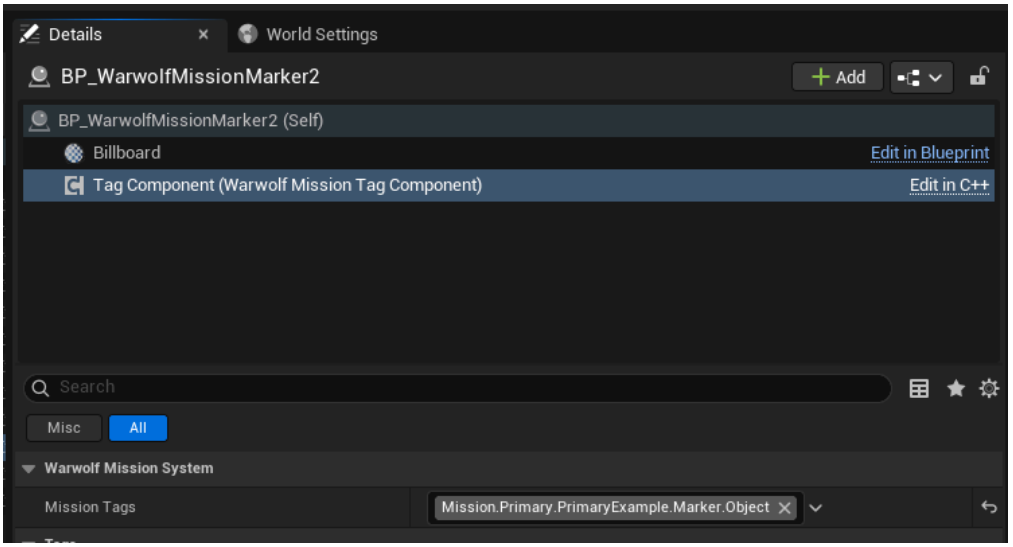


And then on its tag component:



h. For the object's properties and tag component properties:

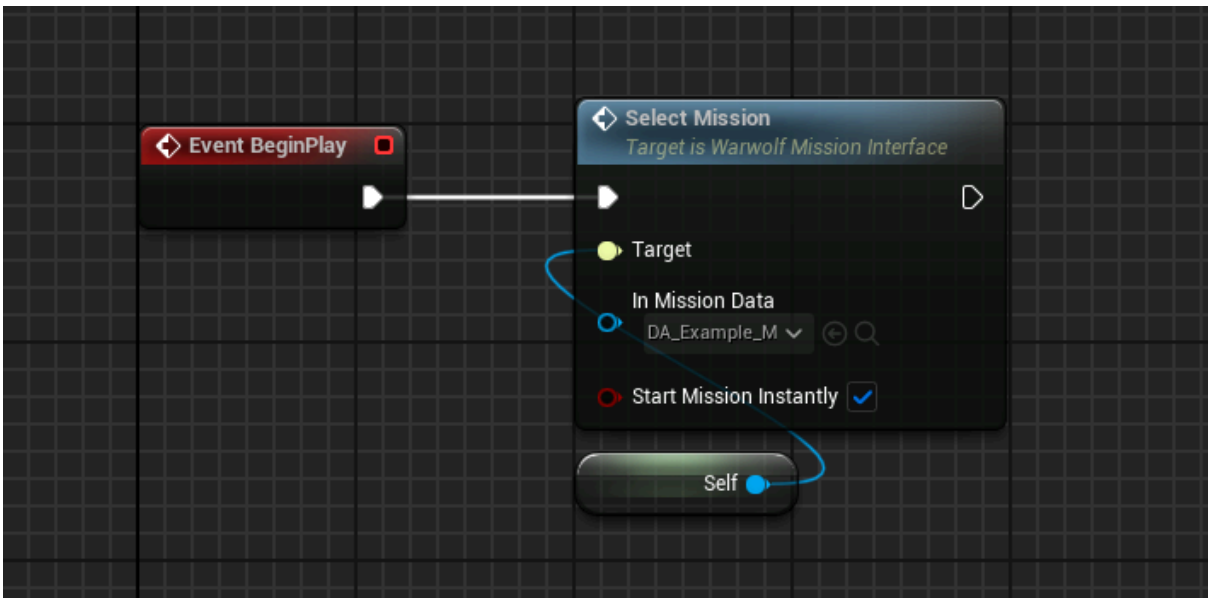




6. Try repeating these steps for yourself with the optional objective. Remember what tags you set up for the optional objective. A hint: You only need one trigger and two markers.

Running the mission

In your game state, set up the following on BeginPlay:

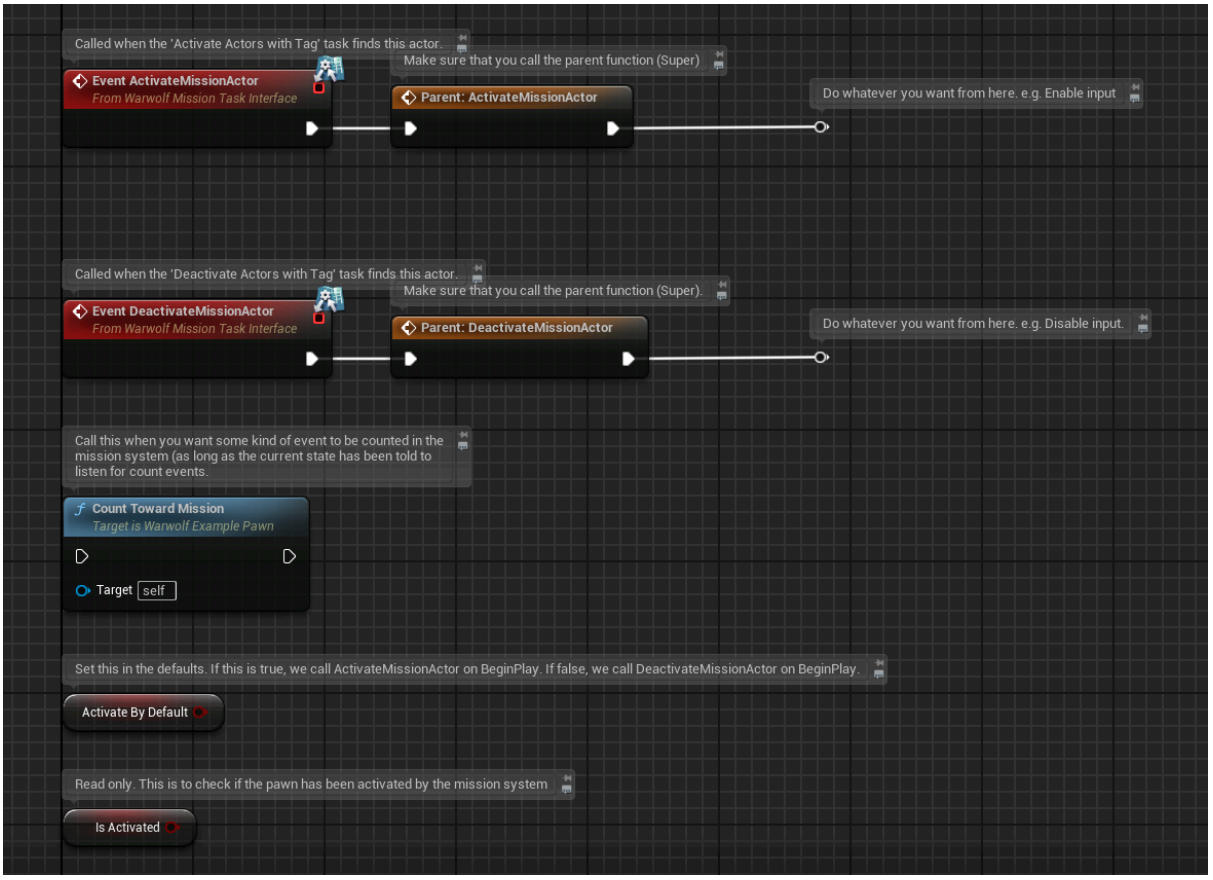


Now, if you want to do more, check out the example project for some UI and more complicated examples of mission setup.

Example Content Breakdown

ExamplePawn/Character/Actor

C++ class: WarwolfExamplePawn, WarwolfExampleCharacter, WarwolfExampleActor
BP class: BP_WarwolfExamplePawn (just to show functions available, just create a new class for the others if you need them)



I've quickly created these classes so that you can also count events with the Count Int task with pawns, characters and general actors (e.g. Kill X enemies sub-objectives, pick up X items, etc.), as the delegate for counting cannot be exposed to blueprints.

Game Mode

C++ class: WarwolfExampleGameMode
BP class: BP_WarwolfGameMode

There's nothing too special about the game mode, only that it has our custom game state set to it.

Game State

C++ class: WarwolfExampleGameState
BP class: BP_WarwolfGameState

Our game state is where we handle all of the events coming from the mission system. It's definitely not the *best* way to do it, but having one place that we know is easily replicated and owned by the server is ideal for our purposes.

You can find a better blueprint breakdown within the tutorial to show you how to get things set up. In the actual example, everything is written in C++ which also replicates things such as UI and Sequence events via multicast.

Cone Door

C++ class: Actor
BP class: BP_ConeDoor

This is just to show that not everything has to be built in C++ to interact with the mission system. Anything with the correct interface and tag component can easily become an actor that can interact/be interacted with by the mission system.

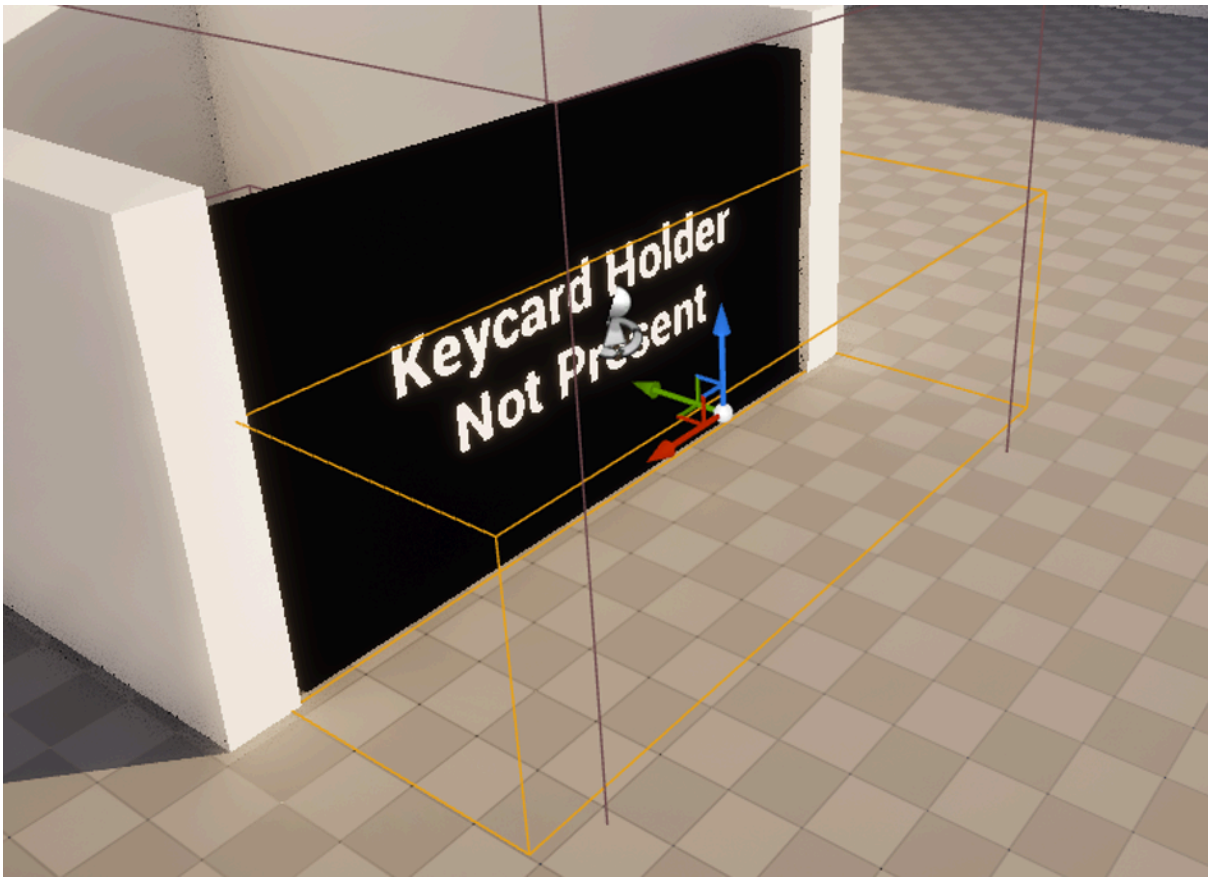
It's a simple cube and trigger volume. The cube moves up and down if the player triggering it has the keycard. The trigger not only handles the door but also sends out mission events whenever triggered (naturally, with the way the objective state tree has been set up, it does not matter that we're firing these events every single time).



Mission Trigger

C++ class: Actor
BP class: BP_TestMissionEventTrigger

This is a simple trigger volume that fires a mission event whenever a pawn overlaps with it. I used this to send mission events when the player had reached certain locations.



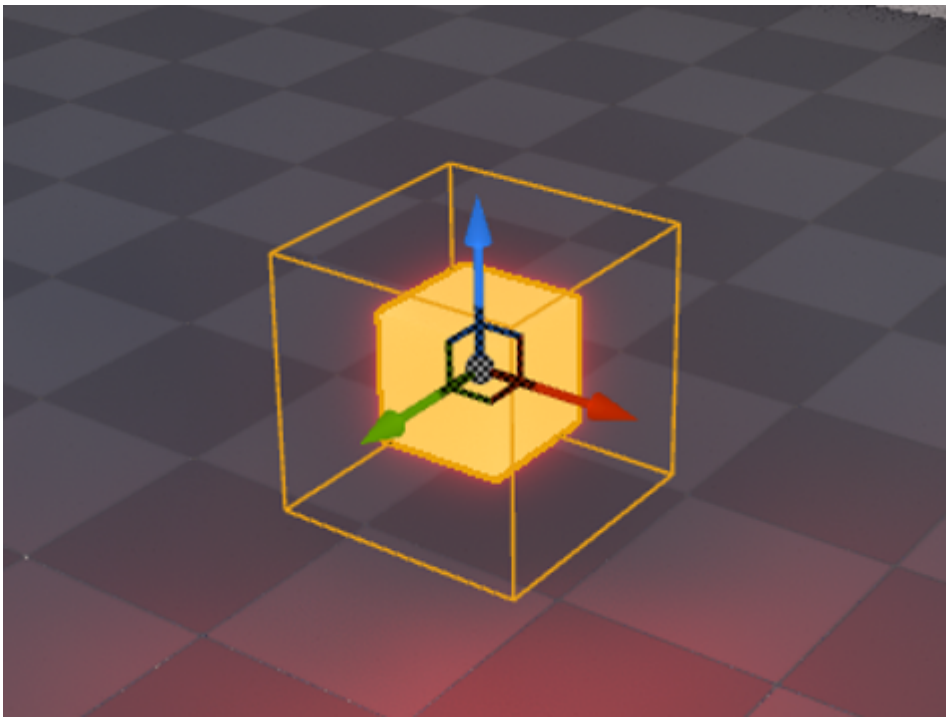
Pickups

C++ class: WarwolfExampleTestPickup
BP class: BP_TestPickup/_Item/_Keycard

This is a base class used for all pickups in the example. The C++ side of things handled the implementation of the component and interface side of things. This base class just changed materials based on its de/activation.

Anything with a suffix to this was the same but had slightly different meshes.

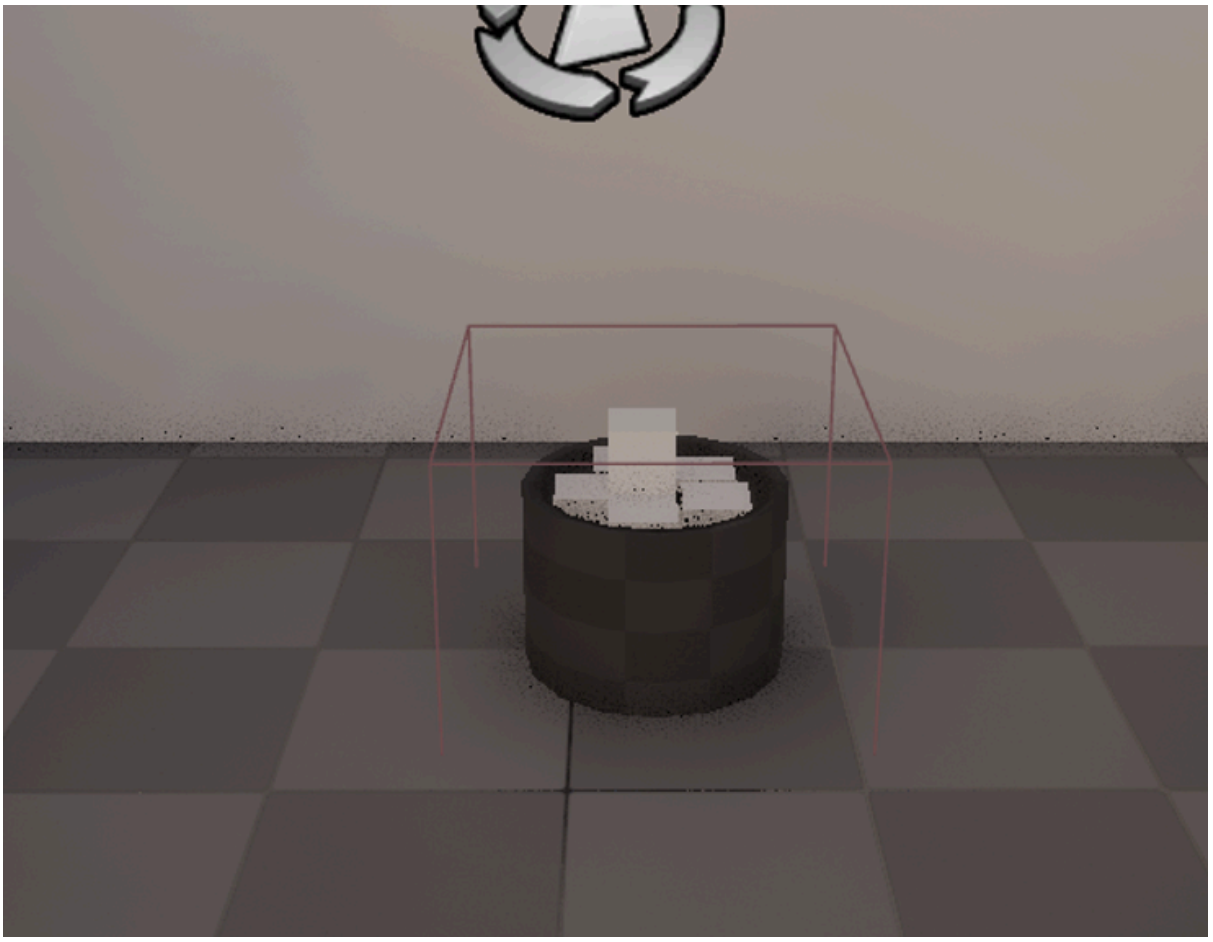
On the _Item variant specifically, we also showcase that the int counter task can be used to count how many of something we count with the mission system.



Item Basket

C++ class: WarwolfExampleTestPickup
BP class: BP_WarwolfPickupBasket

Using the pickup code base class is also meant to showcase that we can create all different kinds of mission-related objects.



HUD

C++ class: WarwolfExampleHUD
BP class: BP_WarwolfHUD

The HUD class handles all of the UI events.

Player State

C++ class: WarwolfExampleHUD
BP class: BP_WarwolfHUD

There's nothing too special about the player state, only that it holds a bool to see if the player has a keycard.

Mission Widget

C++ class: WarwolfExampleMissionWidget
BP class: WBP_WarwolfMission

This is the widget where we contain all of our mission-related UI. Everything is broken off into its own widget so that it's easier to maintain code, and for things like objectives/sub-objectives, we can have as many as we want at once as the state tree can support as many as we need.



Objective Container Widget

C++ class: WarwolfExampleMissionContainerWidget
BP class: WBP_WarwolfMissionContainer

This is just a simple container (i.e. just a vertical box) where we can create a list of objectives.

Objective Widget

C++ class: WarwolfExampleMissionObjectiveWidget
BP class: WBP_WarwolfMissionObjective

This is the widget where our objective is displayed. We can set the name and have a vertical box listing arbitrary numbers of sub-objectives underneath.
Importantly, we assign an FGuid to this so that we're able to track it from the mission system and control it as we please (e.g. showing/hiding).

Sub-Objective Widget

C++ class: WarwolfExampleMissionSubObjWidget
BP class: WBP_WarwolfMissionSubObj

This widget is for displaying sub-objectives. We just show whatever the task for showing sub-objectives asked it to show.

Importantly, we assign an FGuid to this so that we're able to track it from the mission system and control it as we please (e.g. showing/hiding).

Toast Widget

C++ class: WarwolfExampleMissionToastWidget
BP class: WBP_WarwolfMissionToast

This widget shows important updates about the mission, such as the success or failure of an objective and/or mission.

String Table

File Name: ST_Mission

String table that holds all of our mission-related strings. I've just included this for the sake of cleanliness.

Level

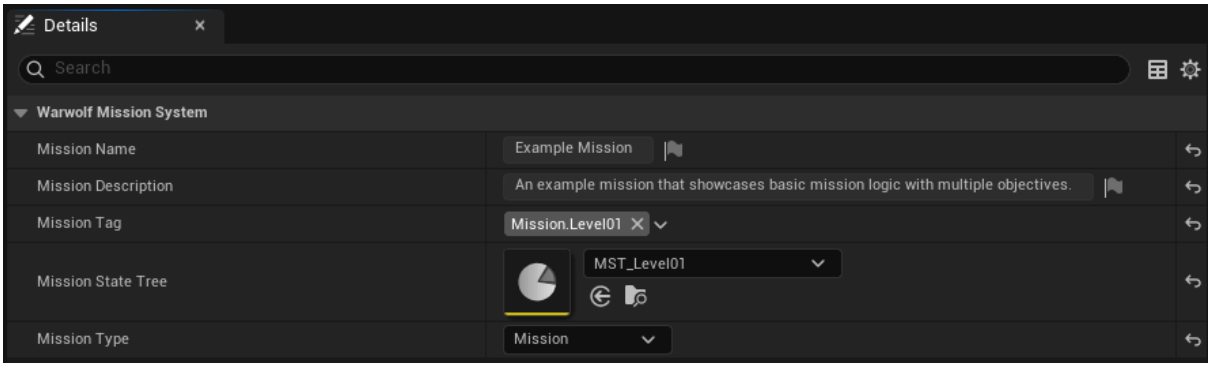
File Name: L_MissionExample

This level includes all of the example content so that you can see how it's being used for a mission. Naturally, this is just a guideline, and you should find your way to set things up in levels to fit your own needs.

Mission Data Asset

File Name: DA_M_Level01

This is the data asset that contains all of the mission.



Mission State Tree

File Name: MST_Level01

A quick breakdown of each state:

MissionWait

We use this state to indefinitely delay the mission logic until we specifically ask it to start via a mission start event.

PlayIntroSequence

Since there's a play sequence task, this is just to showcase the task. The mission logic will begin once the sequence has finished.

MissionStart

This is where we toast the mission beginning, and delay moving onto the next task until the toast has finished displaying.

MissionLogic

This is where all of the objective logic happens. I have added the MissionFinishSuccess and MissionFinishFailed states as children because we want to complete the mission if the primary objective is complete, but still allow the secondary objective's logic to continue to run. If it is not done this way and the MissionFinishSuccess/Failed states are just siblings, any unfinished objectives will automatically stop as we have exited the MissionLogic state.

MissionRunning

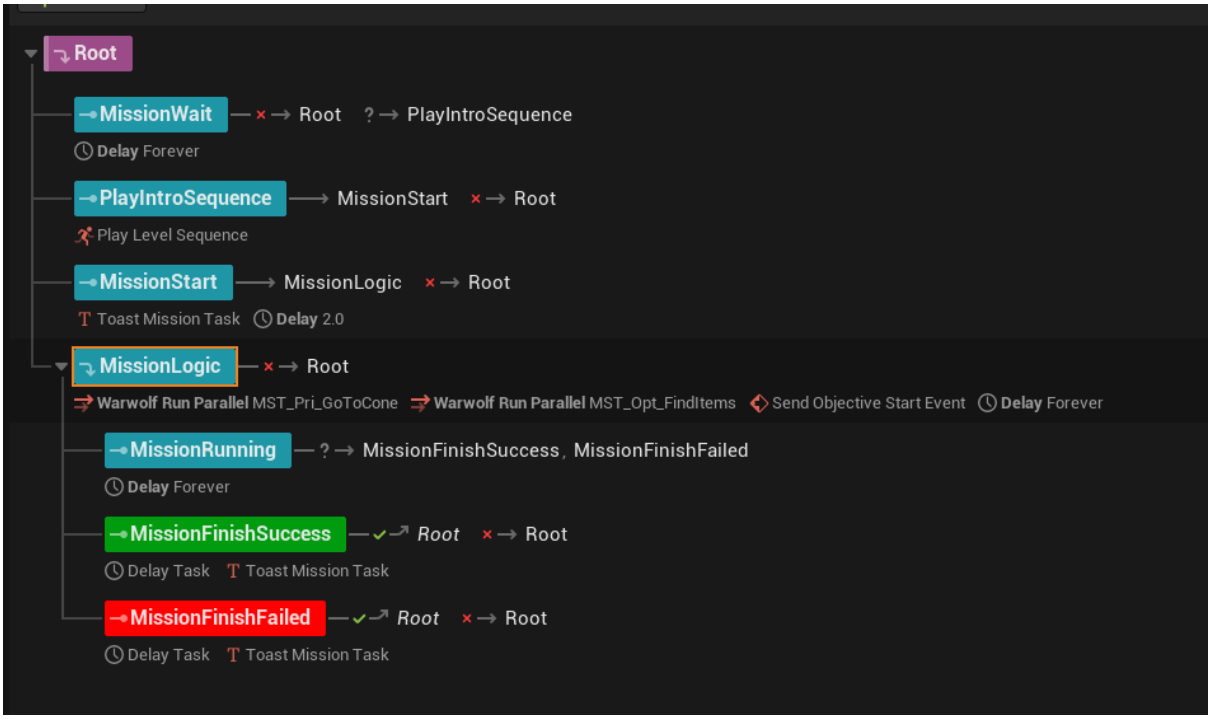
This is just a placeholder state that delays indefinitely so that we stay there until we want to finish the mission in the state tree.

MissionFinishSuccess

We toast that the mission has finished successfully.

MissionFinishFailed

We toast that the mission has finished unsuccessfully.


Root
MissionWait
Root
PlayIntroSequence
Delay Forever
PlayIntroSequence
MissionStart
Root
Play Level Sequence
MissionStart
MissionLogic
Root
Toast Mission Task
Delay 2.0
MissionLogic
Root
Warwolf Run Parallel MST_Pri_GoToCone
Warwolf Run Parallel MST_Opt_FindItems
Send Objective Start Event
Delay Forever
MissionRunning
MissionFinishSuccess, MissionFinishFailed
Delay Forever
MissionFinishSuccess
Root
Root
Delay Task
Toast Mission Task
MissionFinishFailed
Root
Root
Delay Task
Toast Mission Task

Technical Documentation

Plugin

Warwolf Mission Types

File Name: WarwolfMissionTypes.h

A types class for common variables etc. that can be used plugin-wide.

Interfaces

Warwolf Mission Task Interface

File Name: WarwolfMissionTaskInterface.h

This interface is for interacting between mission state tree tasks and the mission system.

Functions

ActivateMissionActor
Activate this actor. (Call from task, implement on an actor.)

DeactivateMissionActor
Deactivate this actor. (Call from task, implement on an actor.)

GetIntCountDelegate
Get a delegate reference for classes that expect to increment an int in a state tree task.

Return: a reference to a delegate we broadcast whenever we expect an int to increment.

> ⚠️This is not accessible for blueprint users. Please use the provided WarwolfMissionPickup class and call the PickedUp function to call this delegate if you work solely in blueprints.

Warwolf Mission Interface

File Name: WarwolfMissionInterface.h

This interface is for interacting between the mission state tree and the mission system.

Functions

SelectMission
Select a mission based on the mission data given.
In Param: InMissionData - Data asset of mission we want to load.
In Param: bStartMissionInstantly - Should we begin the mission right away or wait until it's started manually?

StartMission
Start the mission logic if it hasn't already been started.

SendMissionEvent
Send an event to the mission state tree.
In Param: Event - State tree event to send to the mission system.

Warwolf Mission Tag Interface

File Name: WarwolfMissionTagInterface.h

This interface is for interacting between mission-related logic and the mission system.

Functions

GetMissionTags
Get the mission-related gameplay tags from this actor.
Return: Gameplay tag containers for this owner actor.

Warwolf Mission UI Interface

File Name: WarwolfMissionUiInterface.h

This interface is for interacting between UI and the mission system.

Functions

ToastMission
Show the player that an objective has started.
Param: InMissionName - Name of the mission to show.
Param: InMissionType - Type of mission we're displaying.
Param: InMissionIndex - Index of the mission we're updating.
Param: InToastType - The type of toast we want to display

ShowObjective
Show the player an objective
Param: InObjText - Text to show on the obj widget.
Param: InGuid - The guid we want to assign the objective widget.

HideObjective
Show the player an objective
Param: InGuid - The guid of the obj widget we want to hide.

ShowSubObjective
Hide the given objective.
Param: InSubObjText - Text to show on the sub-obj widget.
Param: InObjGuid - Guid of the objective widget we want to add to.
Param: InGuid - The guid we want to assign the sub-objective widget.

HideSubObjective
Hide given sub-objective.
Param: InObjGuid - Guid of the objective widget we're trying to remove this sub-obj from.
Param: InGuid - The guid of the sub-obj widget we want to hide.

UpdateSubObjective_IntCount
Update a sub-objective with the passed guid with counted ints.
Param: InObjGuid - Guid of the objective widget containing the sub-obj.
Param: InGuid - The guid of the sub-obj widget we want to update.
Param: InCurrentCount - Currently counted int
Param: InTotalCount - Total count for the int.

UpdateSubObjective_Timer
Update a sub-objective with the passed guid with counted ints.
Param: InObjGuid - Guid of the objective widget containing the sub-obj.
Param: InGuid - The guid of the sub-obj widget we want to update.
Param: InCurrentTime - Currently counted time
Param: InTotalTime - Total time.

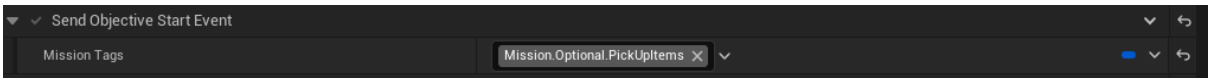
Tasks

Send Objective Start Event

File Name: WarwolfMissionActivateObjSTT.h
Display Name: Send Objective Start Event

A task that sends objective start events for the given objective tags. Useful when you want to delay secondary objectives until a certain point in another mission, or perhaps delay its start until after the primary objective has been activated.

Params



MissionTags
Tags of missions to start. Do not include .Start leaf tag. Only the parent mission tag.

Activate All Markers With Tags

File Name: WarwolfActivateMarkersWithTagSTT.h
Display Name: Activate All Markers With Tags

A task that looks for AWarwolfMissionMarker actors with tags specified in task data, and activates them.

Params



TagsToFind
Gameplay tags to look for. Any AWarwolfMissionMarker or BP_WarwolfMissionMarker inherited class with these tags will be activated.

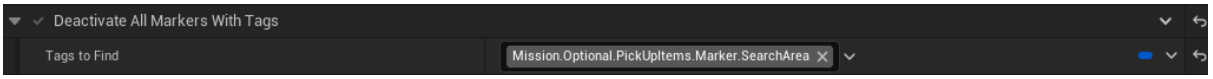
The plugin includes a ready-made BP_WarwolfMissionMarker that you can place in the level with its own widget containing a marker name and distance text.

Deactivate All Markers With Tags

File Name: WarwolfDeactivateMarkersSTT.h
Display Name: Deactivate All Markers With Tags

A task that looks for AWarwolfMissionMarker actors with tags specified in task data, and deactivates them.

Params



TagsToFind
Gameplay tags to look for. Any AWarwolfMissionMarker or BP_WarwolfMissionMarker inherited class with these tags will be deactivated.

The plugin includes a ready-made BP_WarwolfMissionMarker that you can place in the level with its own widget containing a marker name and distance text.

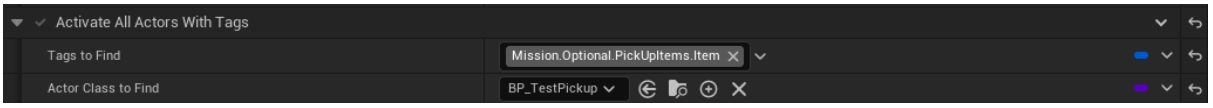
Activate All Actors With Tags

File Name: WarwolfMissionActivateActorSTT.h
Display Name: Activate All Actors With Tags

> ⚠️Requires an actor class to be set up with the gameplay tag component, and inherit the mission interface.

A task that looks for actors of the given class with given tags and activates them.

Params



TagsToFind
Look for any actor containing a Warwolf Mission Tag Component with these tags.

ActorClassToFind
Class of actors to look for. Best to avoid AActor as it'll get expensive.

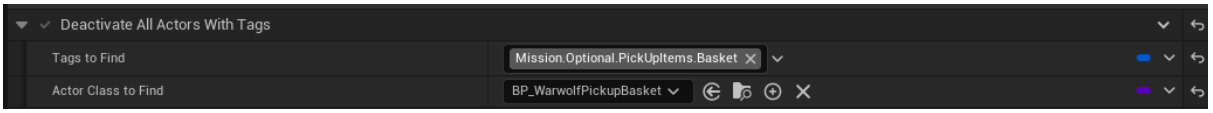
Deactivate All Actors With Tags

File Name: WarwolfMissionDeactivateActorSTT.h
Display Name: Deactivate All Actors With Tags

> ⚠️Requires an actor class to be set up with the gameplay tag component, and inherit the mission interface.

A task that looks for actors of the given class with given tags and deactivates them.

Params



TagsToFind
Look for any actor containing a Warwolf Mission Tag Component with these tags.

ActorClassToFind
Class of actors to look for.

> ⚠️Best to avoid looking for AActor as it'll be expensive!

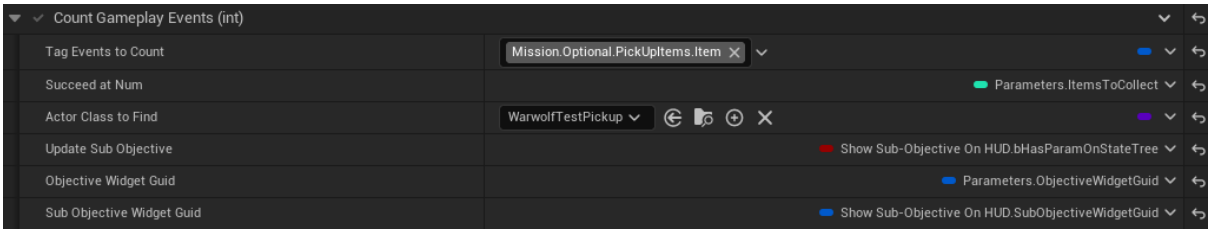
Count Gameplay Events (int)

File Name: WarwolfMissionIntCounterSTT.h
Display Name: Count Gameplay Events (int)

A task that counts an int whenever FOnWarwolfIntCount is broadcast from actors with the given tag.

This is the most complex task in the plugin, even though it's a simple premise.

Params



TagEventsToCount
When we search for actors we should be counting, which gameplay tag should they also have?

SucceedAtNum
How many counts do we succeed this task at? e.g. If we want the player to pick up 4 items before moving onto the next state, set this to 4.

ActorClassToFind
What class of actor should we look for when wanting to bind to counting events?

> ⚠️Best to avoid looking for AActor as it'll be expensive!

bUpdateSubObjective
Do we want to update a sub-objective when something is counted?

ObjectiveWidgetGuid
Reference to the ObjectiveWidgetGuid parameter in the state tree that we want to update with counts. Only needs to be set if bUpdateSubObjective is true.

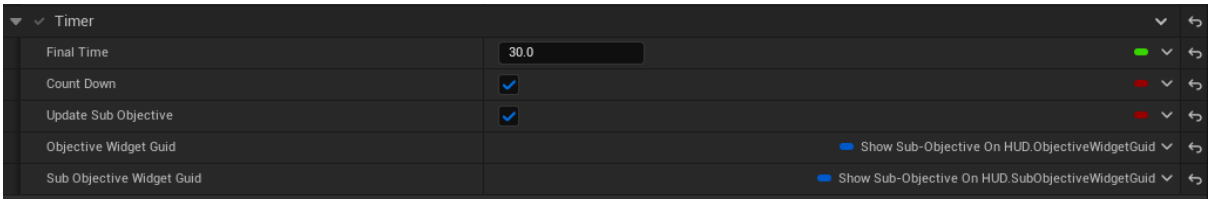
SubObjectiveWidgetGuid
Reference to the ObjectiveWidgetGuid parameter in the state tree that we want to update with counts. Only needs to be set if bUpdateSubObjective is true.

Timer

File Name: WarwolfMissionTimerSTT.h
Display Name: Timer

A task that counts up in seconds on tick until we reach a final value. Useful for timed states.

Params



FinalTime
How many seconds do we count to?

bCountDown
Do we want to count down from the final time instead of up to it? (Only affects UI) e.g. If the time since the task started is 1s and the final time is 10, false will display 1/10, true will display 9/10.

bUpdateSubObjective
Do we want to update a sub-objective when something is counted?

ObjectiveWidgetGuid
Reference to the ObjectiveWidgetGuid parameter in the state tree that we want to update with counts. Only needs to be set if bUpdateSubObjective is true.

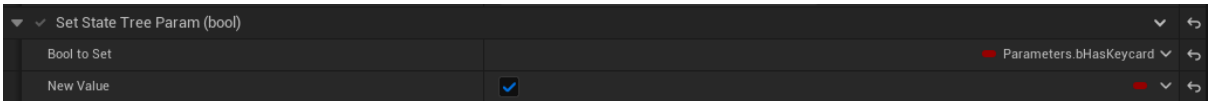
SubObjectiveWidgetGuid
Reference to the ObjectiveWidgetGuid parameter in the state tree that we want to update with counts. Only needs to be set if bUpdateSubObjective is true.

Set State Tree Param (bool)

File Name: WarwolfMissionSetStBoolSTT.h
Display Name: Set State Tree Param (bool)

A task that sets a bool in the state tree parameters.

Params



BoolToSet
Reference to the state tree bool parameter.

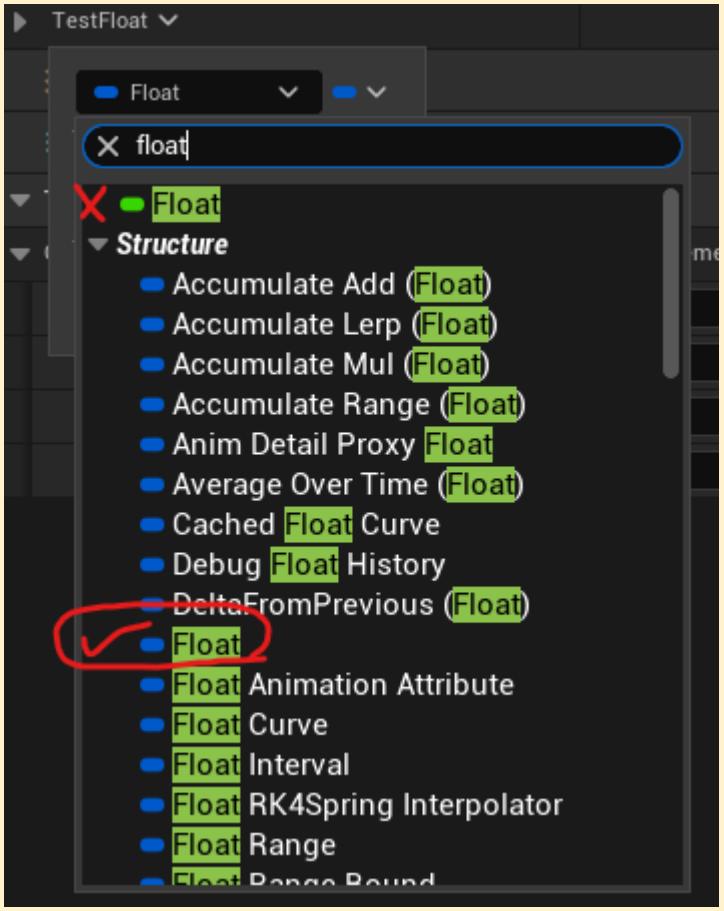
bNewValue
What to set the bool as.

Set State Tree Param (float)

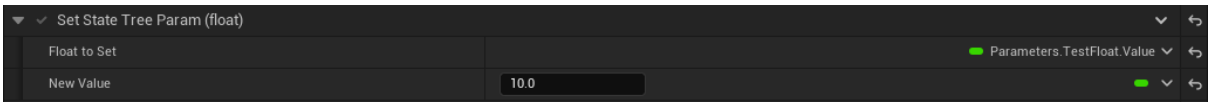
File Name: WarwolfMissionSetStFloatSTT.h
Display Name: Set State Tree Param (float)

A task that sets a float in the state tree parameters.

⚠️Does not use a regular blueprint float, but rather a ‘Float Constant’. You can find this when searching for float in the value type.



Params



FloatToSet
Reference to the state tree float parameter.

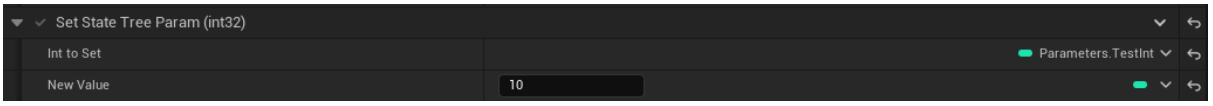
NewValue
What to set the float as.

Set State Tree Param (int32)

File Name: WarwolfMissionSetStInt32STT.h
Display Name: Set State Tree Param (int32)

A task that sets an int32 in the state tree parameters.

Params



IntToSet
Reference to the state tree int32 parameter.

NewValue
What to set the int as.

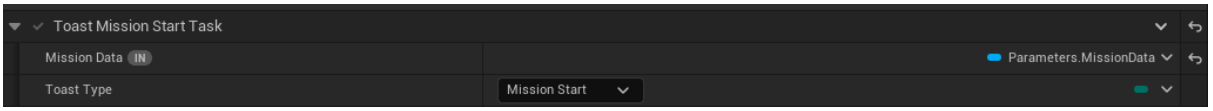
Toast Mission Task

File Name: WarwolfMissionToastStTask.h
Display Name: Toast Mission Task

> ⚠️ Requires UI set-up. Refer to the example project.

A task that toasts a mission on the UI. Use when toasting mission and objective updates.

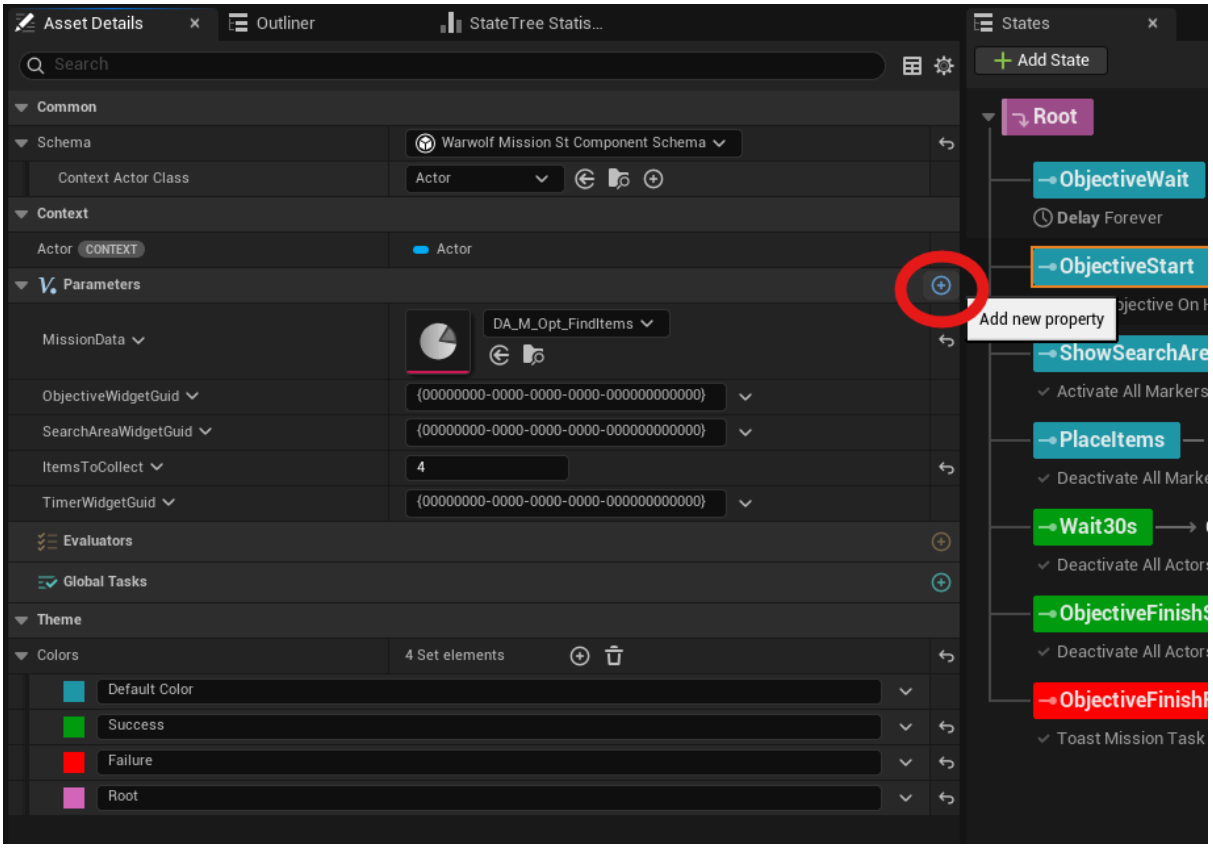
Params

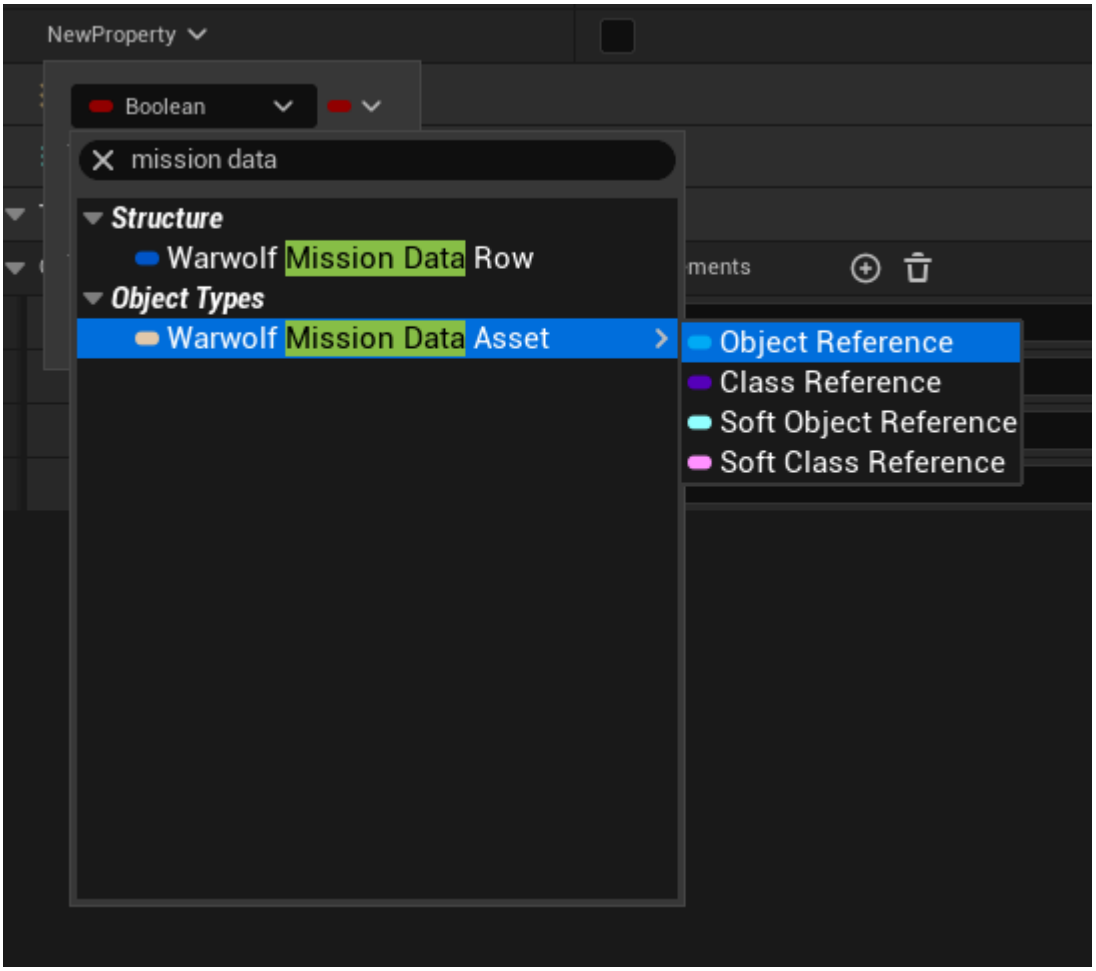


MissionData
The mission data from the state tree parameters.

This is the data asset we use to hold the mission data (mission state tree, name, description, etc.).

Create a state tree parameter and assign it to the task parameter to keep it accessible to all states within the tree.





Toast Type
The type of toast we want to display.

Your choices are:

- Mission Start
- Mission Updated
- Mission Completed
- Mission Failed
- Objective Start
- Objective Updated
- Objective Completed
- Objective Failed

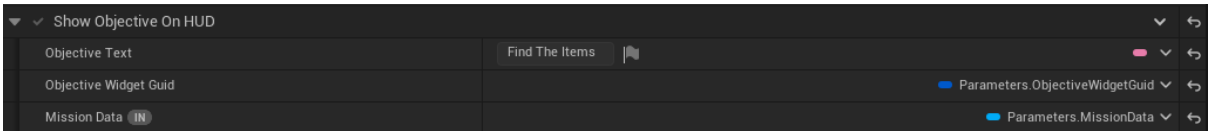
Show Objective On HUD

File Name: WarwolfShowObjectiveUiSTT.h
Display Name: Show Objective On HUD

> ⚠️ Requires UI set-up. Refer to the example project.

A task that shows an objective when entering the state. To be used when an objective has begun.

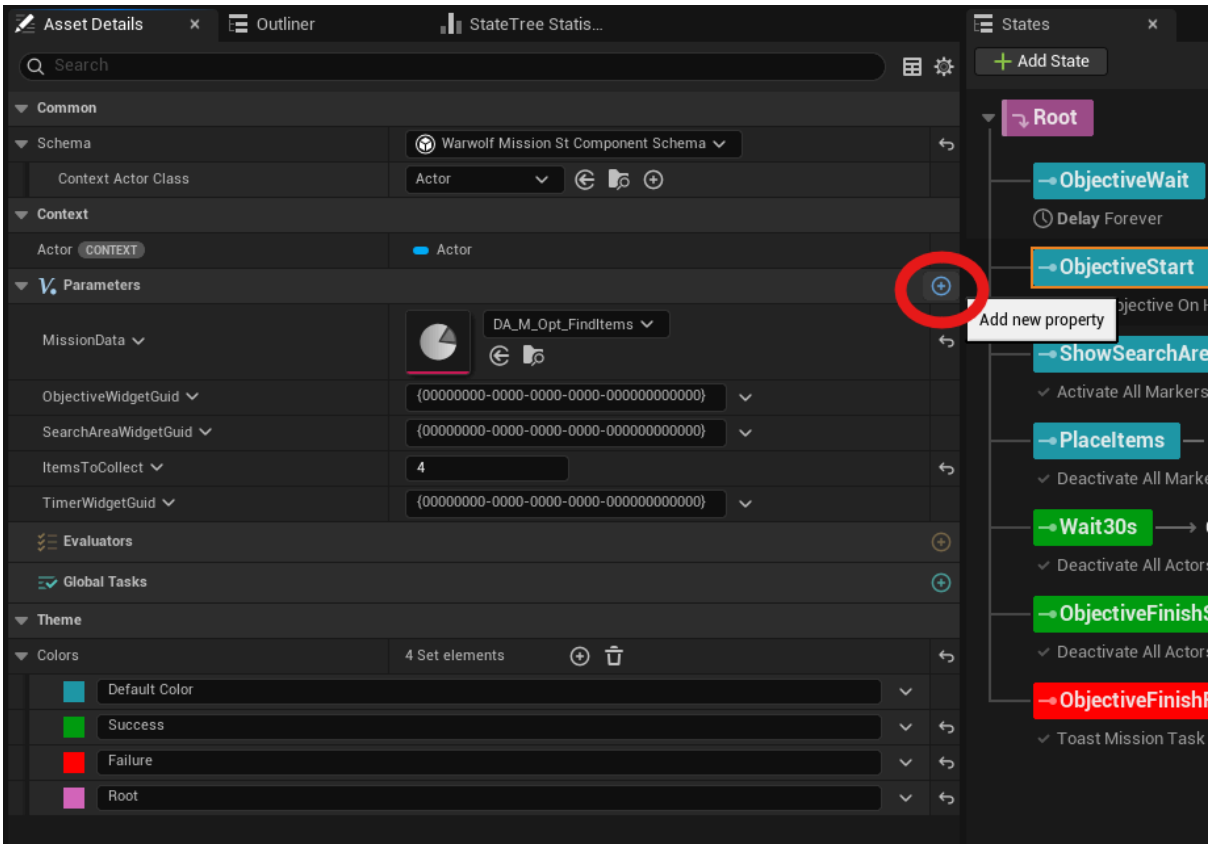
Params

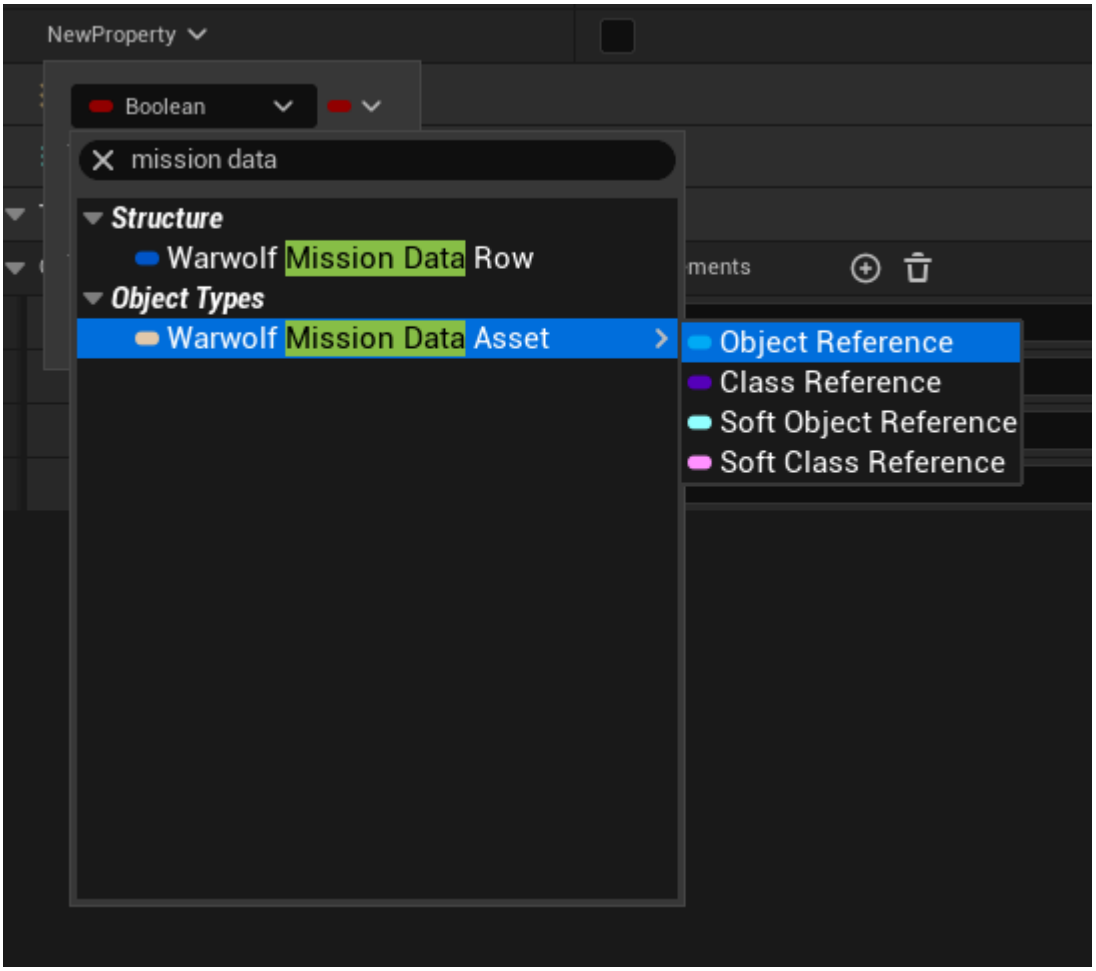


MissionData
The mission data from the state tree parameters.

This is the data asset we use to hold the mission data (mission state tree, name, description, etc.).

Create a state tree parameter and assign it to the task parameter to keep it accessible to all states within the tree.





ObjectiveText
Text we want to display on the objective widget.

ObjectiveWidgetGuid
Reference to the ObjectiveWidgetGuid parameter in the state tree.

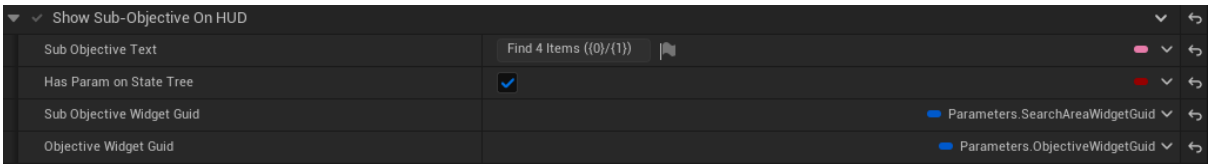
Show Sub-Objective On HUD

File Name: WarwolfShowObjectiveUiSTT.h
Display Name: Show Sub-Objective On HUD

> ⚠️ Requires UI set-up. Refer to the example project.

A task that shows a sub-objective when entering the state, and hides it once the state has finished. Automatically removes itself from the HUD when the state has finished. If you want to keep this up, use nested states.

Params



SubObjectiveText
Text we want to display on the sub-objective widget.

bHasParamOnStateTree
Does this sub-objective need a parameter on the state tree? (e.g. so a counting task can access it)

SubObjectiveWidgetGuid
Reference to this Sub-Objective Widget Guid parameter in the state tree. Useful for when you need to access the widget from another task (e.g. a timer).

ObjectiveWidgetGuid
Reference to the ObjectiveWidgetGuid parameter in the state tree.

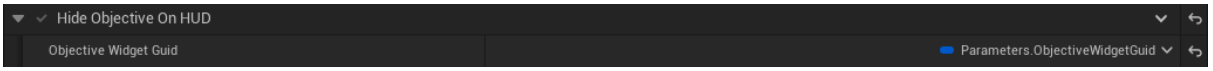
Hide Objective On HUD

File Name: WarwolfMissionHideObjWidgetSTT.h
Display Name: Hide Objective On HUD

> ⚠️ Requires UI set-up. Refer to the example project.

A task that hides an objective widget when entering the state. To be used when an objective is no longer expected to execute any logic (e.g. from a fail or success state).

Params



ObjectiveWidgetGuid
Reference to the ObjectiveWidgetGuid parameter in the state tree.

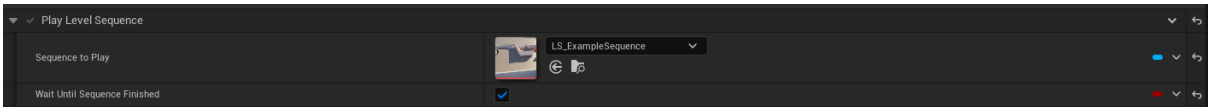
Play Level Sequence

File Name: WarwolfMissionPlaySequenceSTT.h
Display Name: Play Level Sequence

> ⚠️ Requires game state set-up. Refer to the example project.

A task that interface calls to the game state asking it to play a sequence.

Params



SequenceToPlay
What sequence do we want to play?

bWaitUntilSequenceFinished
Should we succeed the task when the sequence is complete? If true, we wait to succeed the task. If false, treat this task as a fire and forget to play a sequence.

Warwolf Mission Task Helper

File Name: WarwolfMissionTaskHelper.h

A utility class to manage dynamic delegate bindings for level sequence playback.
This helper is necessary because dynamic delegates (e.g., OnFinished) in Unreal Engine require a UObject-based target and do not directly support binding to const functions.

This class is an intermediary, forwarding the delegate call to the associated task instance and ensuring compatibility with const methods in StateTree tasks.

Functions

OnLevelSequenceFinished
Handles the delegate call when the level sequence finishes playing.

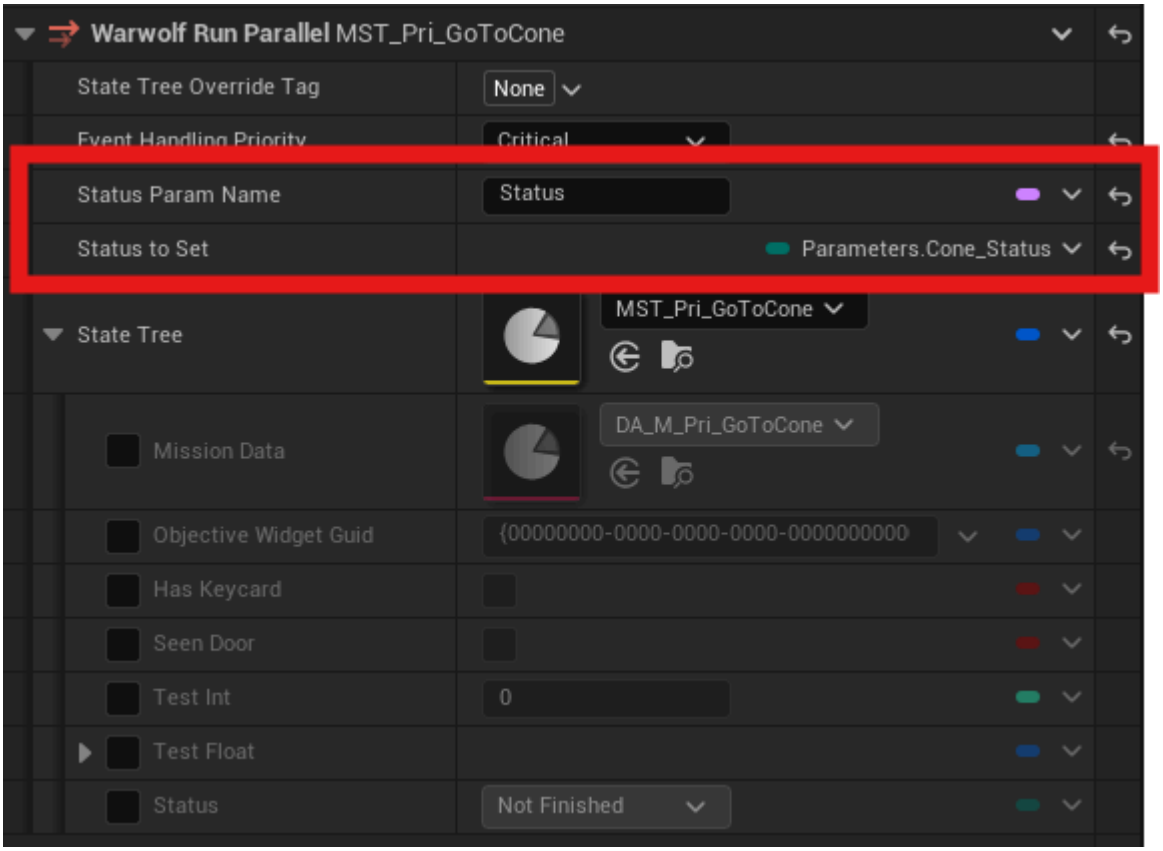
Warwolf Run Parallel

File Name: WarwolfMissionParallelSTT.h

Extension of the 'Run Parallel State Tree' task that also updates the current objective status to a parameter in the state tree running this task.

> ⚠️ Binding properties from the parallel task does not seem to update, even with bCopyBoundPropertiesOnTick being true. Maybe I'm just misunderstanding things. This workaround is the only way I could think of to get around the issue.

Params



StatusParamName
Name of the state tree objective status parameter on the child state tree, e.g. 'CompletionStatus'.

StatusToSet
Reference to the parent state tree parameter for this objective's status.

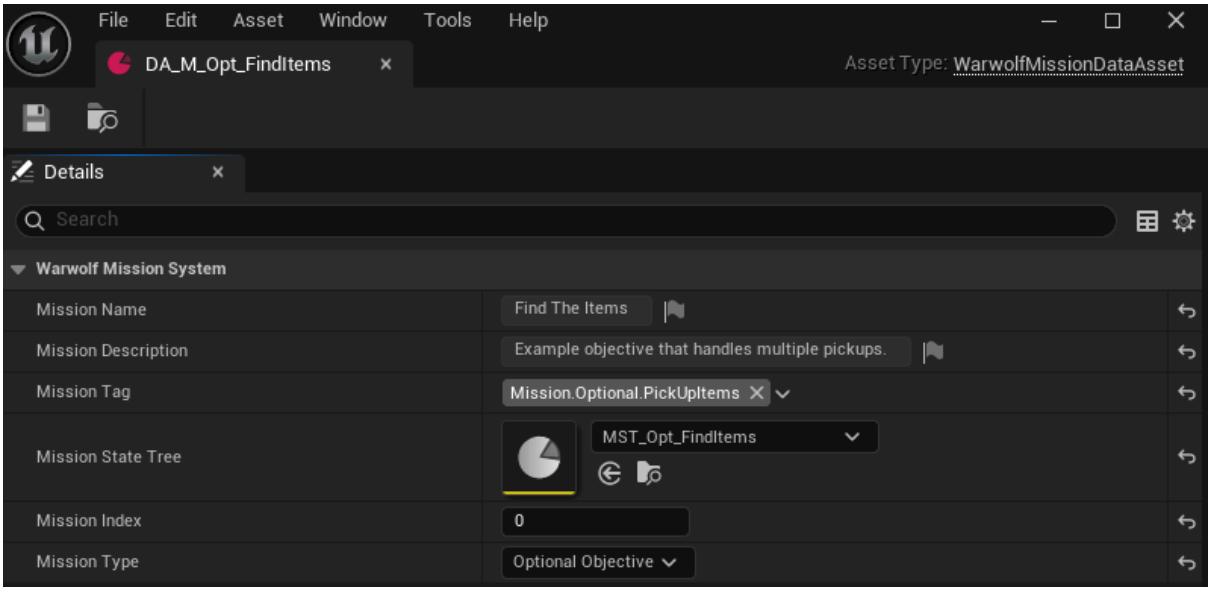
Gameplay Classes

Warwolf Mission Data Asset

File Name: WarwolfMissionDataAsset.h

Data asset for holding all mission information.

Params



Mission Name
Name of the mission. Intended to be shown on the UI, so ideally use localised text.

Mission Description
Description of the mission. Not explicitly used in the example project, but useful for displaying on a mission select screen, or just context for designers.

Mission Tag
Gameplay tag associated with the mission.

Mission State Tree
State tree used for the mission. This should not be an objective state tree ideally, as the mission tree has some tasks that would make it difficult to otherwise re-use that objective tree in other missions.

Mission Type
The type of mission/objective this is.

Options are:

- Mission
- Primary Objective

- Optional Objective

Warwolf Mission Manager

File Name: WarwolfMissionManager.h

State tree component responsible for controlling mission logic. Should be placed on the Game State.

Functions

SendMissionEvent
Overridden from IWarwolfMissionInterface.

SetMissionStateTree
Set the state tree to use for this mission.
In Param: InStateTree - State tree to run.

Warwolf Mission Marker

File Name: WarwolfMissionMarker.h
Blueprint: BP_WarwolfMissionMarker for a ready-made solution.

Actor class responsible for displaying a widget where a designer wants. Supports activation/deactivation from state tree tasks.

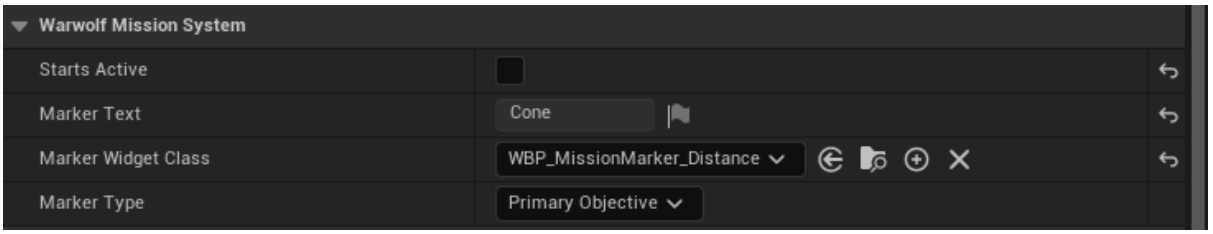


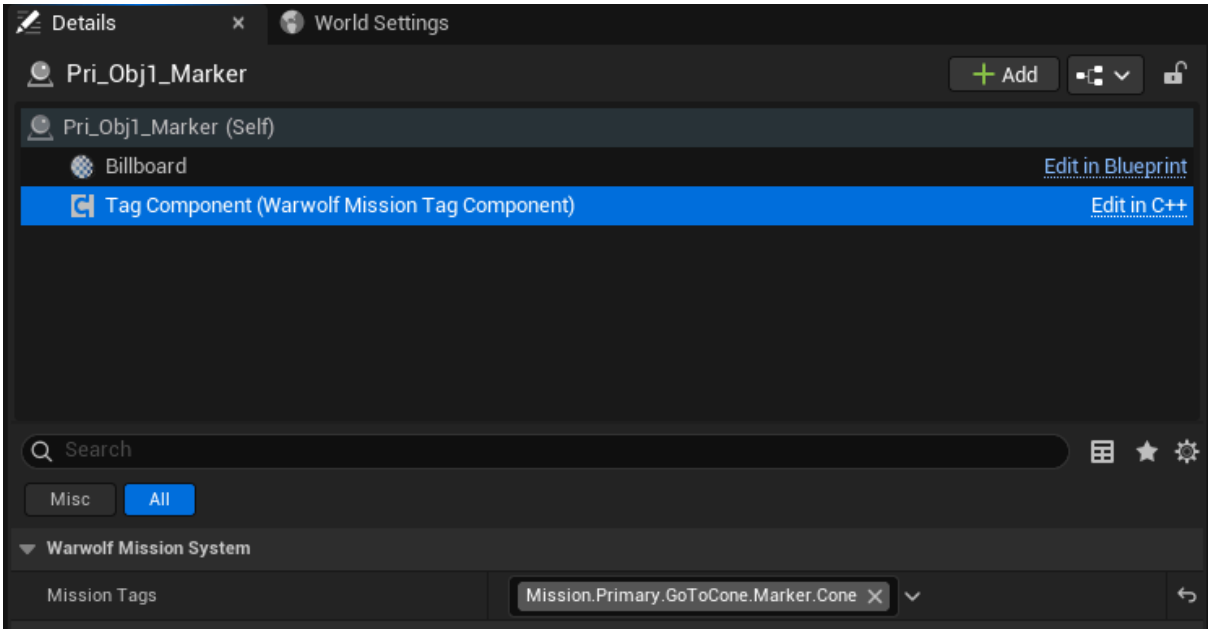
Functions

Activate Marker
Activate this marker and show it on the HUD.

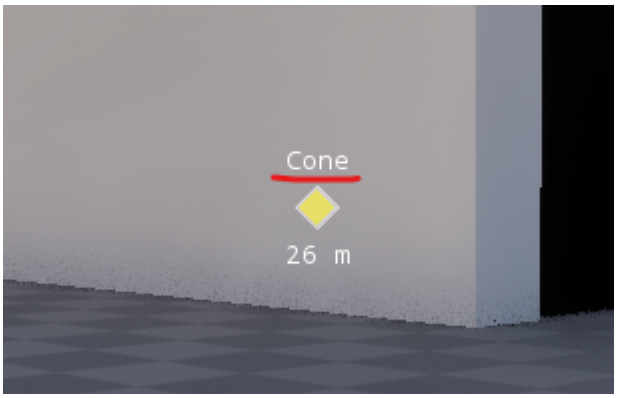
Deactivate Marker
Deactivate this marker and hide it on the HUD.

Params





Marker Text
Text to display on the marker.



Marker Widget Class
Class of the widget we want to use for this marker. i.e. its appearance.

This shouldn't need changing if you use the blueprint mission marker until you want to use your own widget class.

Marker Type
What type of objective is this marker used for?

We use this to change the colour/behaviour of the marker.

Options are:

- Mission
- Primary Objective
- Optional Objective

You shouldn't use the Mission option as it wouldn't ever really be required unless you want to use it as a specific edge case marker with different behaviour.

bStartsActive
Does this marker start active? It will immediately become visible and active if true. Invisible until activated if false.

Warwolf Mission Marker Widget

File Name: WarwolfMissionMarkerWidget.h
Blueprint: WBP_WarwolfMissionMarker_Distance for a ready-made solution.

Native base class for widget that displays when a mission marker is active. Should always have a blueprint layer.



Functions

FindScreenEdgeLocationForWorldLocation
Converts a world location to screen position for HUD drawing. This differs from the results of FSceneView::WorldToScreen in that it returns a position along the edge of the screen for offscreen locations
Thanks OP and ksztanan for this.
https://forums.unrealengine.com/t/easy-offscreen-indicator-blueprint-node/21342/52

In Param: InLocation - The world space location to be converted to screen space
In Param: EdgePercent - How close to the edge of the screen, 1.0 = at edge, 0.0 = at center of screen. .9 or .95 is usually desirable
In Param: ViewportCenterLoc - for offsetting center of the screen, leave at (0.5, 0.5) for no offset
Return: OutScreenPosition - the screen coordinates for HUD drawing
Return: OutRotationAngleDegrees - The angle to rotate a hud element if you want it pointing toward the offscreen indicator, 0° if onscreen
Return: bIsOnScreen - True if the specified location is in the camera view (may be obstructed)

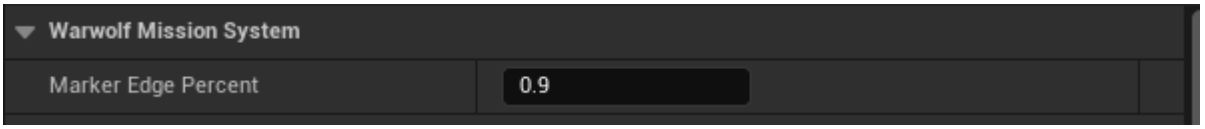
UpdateMarkerName
Update the name of the marker.
In Param: InName - Name this marker will display.

UpdateMarkerDistance
Update distance text based on marker distance to player.
In Param: InDistance - Distance to display on the marker.

UpdateMarkerLocation
Update location of the owning marker as it may be attached to something.
In Param: InLocation - World location of the owning marker.

UpdateMarkerType
Update colour of the marker based on the type of objective it's for.
In Param: InMissionType - Type of mission this marker is used for.

Params



MarkerEdgePercent
At what percent does the marker stop at the edges of the screen? Best left at default for consistency.

Warwolf Mission Component Schema

File Name: WarwolfMissionStComponentSchema.h

State tree schema that we use for the mission system. It is important that mission/objective state trees use this when creating new ones.

Warwolf Mission Tag Component

File Name: WarwolfMissionTagComponent.h

Component we place on any actor we want to interact with the mission system in some way.

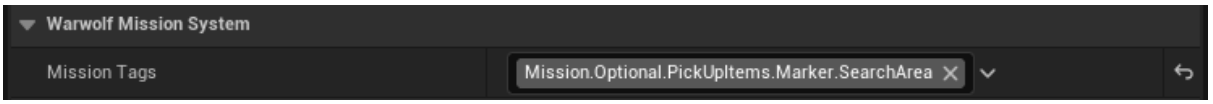
For example, we can place this on a pick-up actor that we want to activate/deactivate only when an objective reaches a certain stage.

Functions

GetMissionTags
Get MissionTags tag container.
Return: MissionTags - The mission tags currently assigned to this component.

SetMissionTags
Set MissionTags tag container.
In Param: InTags - The mission tags we want to assign to this component.

Params



MissionTags
Mission-related tags we want to identify the parent actor by.